From Fields to Dose: Technical Documentation

Author record: https://orcid.org/0009-0005-3459-0859

Internal technical reference for the benchmark software, data formats, scoring rules, analyses, integrity controls and operating procedures.

1. Scope and conventions

This document describes the software behind the study From Fields to Dose: Evaluating Large Language Models for Weight-Based Pediatric Medication Extraction and Dosing Consistency. It covers the repository and its modules, the environment and configuration, every data format read or written by the pipeline, the generation of the synthetic corpus, the lock and the integrity controls built around it, the execution harness, the scoring rules exactly as implemented in the locked scorer, each analysis script with what it found on the four completed models, the figures, the evidence packet, the public release procedure, an operating runbook of the commands actually used, and the known limitations of the implementation. It is written for a reader who needs to reproduce, audit or extend the benchmark, and it is the companion of three other documents: the study report (design, validation, results and deviations), the system architecture document (software, data model, pipeline, operations and release plan) and the feature article (publishable text). Protocol v1.0 is the registered design document on the Open Science Framework.

Sources. Every statement here is taken from the system architecture document, the study report, the verbatim scorer and lock excerpts, the source files of the tools added after registration (analysis/pairwise.py, tools/per_drug.py, tools/admin_amount.py, tools/by_stratum.py, tools/make_public_release.py) and the recorded project timeline. Where a fact is needed and none of those sources records it, this document says "not recorded" rather than supplying a value. Code, docstrings, commands and lock values are quoted verbatim and never composed from memory.

Writing standard. No em or en dashes; no clock times; hyphens only inside compound terms; calendar dates written in full; no author name, only "the author" and the ORCID line under the title; no development version labels for the working documents, the single permitted versioned name being Protocol v1.0. Model names are given as the identifiers used in config/models.yaml and in each raw file's _config line. The repository link appears once, in the closing section of this document.

Terminology used throughout. "Workflow B" is end-to-end extraction with model-computed dose; "workflow C" is extraction with deterministic arithmetic at scoring time. "run1" is the full test-set run of 361 records; "run2" and "run3" are repeat runs on the 108-record subset. "Non-abstention records" are the 304 test records per model and workflow that carry a complete canonical regimen; the remaining 57 are abstention cases. "Composite failure" and "dose_consequential" name the same Level 4 flag. "The seven hashed files" are the files whose concatenated bytes form the lock's scoring hash, listed in Section 7.

2. Design principles

The architecture document states six principles, and every control described later in this document is an implementation of one of them.

1. Nothing that affects a result is editable after the lock without a logged deviation: the test set, the scorer, the calculator, the prompts, the output schema and the registered analysis script are hashed together, and the scorer refuses to run if any of them changed.

2. Every number in the paper is produced by one committed script from committed inputs. No hand computation.

3. Every provider is accessed through the same OpenAI-compatible client with a pinned model identifier; the only per-provider differences are endpoint and key name.

4. Runs are resumable and idempotent at the record level: interrupting and restarting never loses or duplicates a scored record.

5. The evaluated models never share a family with the generator and auditor, enforced in code.

6. All artefacts are releasable: synthetic data, prompts, outputs, logs, configuration without secrets.

Two consequences of these principles shape the rest of the design. First, because the registered analysis script is inside the hashed set, any later correction to the analysis cannot be made in place; it must be implemented as a separate script outside the hashed set with a deviation row, which is exactly what happened on 5 October 2026 (Section 7). Second, because results are produced only by committed scripts, the supplementary tables added after registration (per drug, written amount, stratified, maximum check) are themselves scripts with their own output files, and their outputs are copied into the evidence packet and the public release rather than being transcribed by hand.

3. Repository layout

The working repository is private; a public release repository is regenerated from it by script (Section 13). The layout, as given in the architecture document:

```
f2d/
  config/models.yaml            evaluated models, generator, auditor, budget ceiling, repeat
settings
  sources/                      dosing table, FDA labels, growth references, age bands, source
checks
    dosing_sources.csv          22 verified rows, 21 drugs, label set ids, bases, ranges, maxima
    labels/<drug>/              openFDA label JSON per drug
    wtageinf.csv, wtage.csv     WHO 0 to 24 months and CDC 2 to 20 years LMS tables
    age_bands.json              NICHD bands
    fetch_labels.py, check_sources.py
  generator/                    record synthesis
    build_corpus.py             scenario and record generation, weight sampling, distractors, dates
    templates.py                note templates by complexity level, frequency vocabulary
    calculator.py               deterministic exposure calculator (locked)
```

```
    audit.py                    realism audit with a committed rubric

  harness/                      model execution

    prompts/system.txt, task_B.txt, task_C.txt   committed prompts (locked)

    output_schema.json          committed output object (locked)

    common.py                   client construction, retry policy, JSON extraction, family check

    runner.py                   resumable runner for a split, workflow, run id, optional model and
subset

  scoring/

    score.py                    four-level scoring, abstention scoring, automatic error classes
(locked)

    codebook.md                 manual coding classes and procedure

  analysis/

    analyze.py                  registered primary analysis, clustered bootstrap, hypothesis tests
(locked)

    analyze_malformed_as_failure.py   the registered analysis with the malformed-row correction
(deviation; outside the lock)

    stability.py                run-to-run agreement on the repeat subset

    pairwise.py                 Holm-corrected pairwise model contrasts (added after registration)

  tools/

    lock_testset.py             writes LOCK.json; hash_file and hash_scoring used by the scorer's
check

    self_audit.py, remove_records.py, make_repeat_subset.py

    code_errors.py              blinded manual coding and re-coding

    per_drug.py, admin_amount.py, by_stratum.py   supplementary and exploratory tables (added after
registration)

    export_results.py           export of summary, tables, manifest, lock, deviations, sanitised
config

    ping_models.py, smoke.py, secret_scan.py, show_dosing.py, set_row.py, fetch_query.py

  data/dev/                     development split (172 records)

  data/test/                    locked test split (361 records), LOCK.json, repeat_subset.txt,
audits

  protocol/                     Protocol v1.0 (docx and PDF), LOCK_hashes.txt, DEVIATIONS.md,
README.md, archive/

  results/                      raw and scored run files, analysis outputs, manual codes, logs/,
export/

  tests/                        unit tests (25)

  SETUP_GUIDE.md                BLOCK 0 to BLOCK 8 operating procedure
```

The table below assigns one responsibility to each module and states whether it is inside the lock. "Locked (hash_scoring)" marks the seven files whose concatenated bytes form scoring_sha256 in data/test/LOCK.json. "Locked (LOCK.json data hash)" marks the three data files hashed individually in the same lock file. Everything else can change after registration without breaking the scorer's check, although any change that affects a reported result still requires a dated row in protocol/DEVIATIONS.md.

Module	Responsibility	Locked or not
config/models.yaml	Evaluated models, generator, auditor, budget ceiling, repeat settings; environment-variable names only	Not locked
sources/dosing_sources.csv	22 verified rows for 21 drugs: label set ids, bases, ranges, maxima, notes on derivations	Not locked
sources/labels/<drug>/	One openFDA label JSON per drug	Not locked
sources/wtageinf.csv, sources/wtage.csv	WHO 0 to 24 months and CDC 2 to 20 years LMS tables for weight sampling	Not locked
sources/age_bands.json	NICHD age bands with the two collapsed categories	Not locked
sources/fetch_labels.py	Pulls one label per drug from openFDA with query overrides for salts	Not locked
sources/check_sources.py	Enforces completeness of the dosing table; flags a drug with both per-dose and per-day rows	Not locked
generator/build_corpus.py	Scenario and record generation, weight sampling, distractors, dates	Not locked
generator/templates.py	Note templates by complexity level, frequency vocabulary	Not locked
generator/calculator.py	Deterministic exposure calculator used by workflow C scoring	Locked (hash_scoring)
generator/audit.py	Realism audit with a committed rubric, run by the local auditor model	Not locked
harness/prompts/system.txt	Committed system prompt with the output object example and abstention rule	Locked (hash_scoring)
harness/prompts/task_B.txt	Workflow B task prompt	Locked (hash_scoring)
harness/prompts/task_C.txt	Workflow C task prompt	Locked (hash_scoring)
harness/output_schema.json	Committed output object	Locked (hash_scoring)
harness/common.py	Client construction, retry policy, JSON extraction, family check	Not locked
harness/runner.py	Resumable runner for a split, workflow, run id, optional model and subset	Not locked
scoring/score.py	Four-level scoring, abstention scoring, automatic error classes, lock check	Locked (hash_scoring)
scoring/codebook.md	Manual coding classes and procedure	Not locked
analysis/analyze.py	Registered primary analysis, clustered bootstrap, hypothesis tests	Locked (hash_scoring)
analysis/ analyze_malformed_as_failure.py	The registered analysis with the malformed-row correction	Not locked (deviation; outside the hashed set)
analysis/stability.py	Run-to-run agreement on the repeat subset	Not locked
analysis/pairwise.py	Holm-corrected pairwise model contrasts; H3 on the primary endpoint	Not locked (added after registration)
tools/lock_testset.py	Writes LOCK.json; hash_file and hash_scoring used by the scorer's check	Not locked
tools/self_audit.py, remove_records.py, make_repeat_subset.py	Blinded author self-audit; removal of records; stratified repeat subset	Not locked
tools/code_errors.py	Blinded manual coding and re-coding	Not locked
tools/per_drug.py	Per-drug supplementary table	Not locked (added after registration)

tools/admin_amount.py	Written per-dose amount exploratory table	Not locked (added after registration)
tools/by_stratum.py	Stratified table for the heat maps	Not locked (added after registration)
tools/max_check.py	Post hoc maximum check	Not locked (added after external review)
tools/export_results.py	Export of summary, tables, manifest, lock, deviations, sanitised config	Not locked
tools/ping_models.py, smoke.py, secret_scan.py, show_dosing.py, set_row.py, fetch_query.py	Connectivity check; smoke run; secret scan before any push; dosing table inspection and editing; label query	Not locked
tools/make_public_release.py	Allow-list copy into the public release directory with secret scan	Not locked
data/dev/	Development split (172 records)	Not locked
data/test/records.jsonl	Locked test records (361)	Locked (LOCK.json data hash)
data/test/scenarios.jsonl	Locked test scenarios	Locked (LOCK.json data hash)
data/test/repeat_subset.txt	108-record stratified repeat subset	Locked (LOCK.json data hash)
data/test/LOCK.json	The lock file itself, written once by tools/lock_testset.py	Not hashed; the runner refuses to start without it
protocol/	Protocol v1.0 (docx and PDF), LOCK_hashes.txt, DEVIATIONS.md, README.md, archive/	Not locked; registered on OSF
results/	Raw and scored run files, analysis outputs, manual codes, logs/, export/, packet	Not locked; outputs
tests/	25 unit tests	Not locked
SETUP_GUIDE.md	BLOCK 0 to BLOCK 8 operating procedure	Not locked

4. Environment and configuration

Language and environment. The code is Python 3 and runs from a local working copy under a Python virtual environment on the author's machine. The virtual environment directory is excluded from the public release (Section 13). Unit tests: 25, in tests/, run in BLOCK 1 of the setup guide together with a smoke run.

Secrets. Provider keys live in a git-ignored .env file and nowhere else. config/models.yaml holds environment-variable names only and is therefore safe to publish; the sanitised configuration exported by tools/export_results.py and the copy in the evidence packet carry the same environment-variable names and no values. tools/secret_scan.py is run before any push, including over results and logs, and tools/make_public_release.py runs its own pattern scan over the release directory and refuses to complete if a pattern matches (Section 13). All provider keys are rotated after the study, at acceptance. Two Google key names are visible in the key-swap procedure, GOOGLE_API_KEY and GOOGLE_API_KEY_PROJECT1 (Section 8); the names of the other providers' environment variables are not recorded in the sources used here.

Model configuration. config/models.yaml lists the evaluated models, the generator, the auditor, the budget ceiling and the repeat settings. Every provider is accessed through the same OpenAI-compatible client constructed in harness/common.py, with a pinned model identifier; the only per-provider differences are the endpoint and the key name. harness/common.py also holds the retry policy, the JSON extraction routine (extract_json) and the family check that refuses to evaluate a model sharing a family with the generator or auditor.

Model identifiers as used: gpt-5.5-2026-04-23, claude-sonnet-5, gemini-3.1-pro-preview (OpenAI-compatible endpoint), deepseek-v4-pro, llama3.1:8b (Ollama, local). The GPT-5.5 identifier carries a release date; the Claude and Gemini identifiers name a release; the DeepSeek identifier resolves to the provider's current release at call time and is therefore not immutable. For that reason the run manifest records first and last call timestamps per

run file (all test runs 4 to 5 October 2026) rather than claiming that every provider version was fixed. The local model is served by Ollama; its digest for llama3.1:8b is 46e0c10c039e, recorded in the manifest. The paraphrase generator and realism auditor are qwen2.5:14b, also local, a family distinct from every evaluated model; the harness refuses an evaluated model that shares a family with the generator. Exact identifiers are also written into each raw file's _config line, so a scored result can always be traced to the identifier that produced it.

Budget. The test phase has a 200 USD ceiling in the configuration; spend is recorded per provider in the manifest. Observed per-call cost: GPT-5.5 about 3.1 cents; Gemini about 2.2 cents; Claude about 1.0 cent; DeepSeek about 0.9 cents; Llama local at no provider cost. Approximate test-phase spend from provider dashboards: OpenAI about 37 USD, Google about 28 USD, Anthropic about 13 USD, DeepSeek about 11 USD; the full study is under 100 USD against the ceiling. Final figures from the dashboards go in the run manifest.

5. Data formats

All record-level files are JSON Lines. Supplementary tables are CSV. The field lists below are those given in the architecture document's data model, the verbatim scorer excerpt and the source of the supplementary tools; where a file's exact column set is not in those sources, this is stated.

5.1 Scenario (data/test/scenarios.jsonl). One canonical regimen per line: scenario id; drug; dosing basis (mg/kg/dose, mg/kg/day, mcg/kg/dose, mcg/kg/day); weight-normalised dose; frequency per day; route; concentration; maxima as max_per_dose_mg or max_per_day_mg; expected per-dose amount (printed, rounded, maximum applied) and expected mg/kg per dose and per day; age band; complexity; required fields; missing fields; conflict fields; challenge tags (for example derive_per_dose, multiple_weights, discontinued_same_drug, lb_documentation, multiple_meds, dose_cap, per_day_basis, mcg_drug, weight_tail); abstention flag. Field names visible in the scorer excerpt and tools: scenario_id, drug, challenges, expected_dose_per_admin_mg, expected_mg_per_kg_per_dose, expected_mg_per_kg_per_day, max_per_day_mg, max_per_dose_mg, current_weight_value, current_weight_unit. The study report adds that scenarios carry current and prior weight values with units and timestamps, missing and conflict fields, distractor orders and challenge tags.

5.2 Record (data/test/records.jsonl). One note instance of a scenario: record id (pattern test-<drug>-<basis>-<band>-L<level>-v<variant>-r<round>); scenario id; split; complexity; age band; generation method (template or paraphrase); abstention flag; text. The test split holds 361 records (two variants per scenario after audit removals), of which 57 are abstention cases; the development split holds 172.

5.3 Raw output (results/raw_test_<workflow>_<model>_<run>.jsonl). The first line is a _config object. The runner writes into it the model id, endpoint, workflow, run id, subset, started, system_prompt_sha256 and task_prompt_sha256; the architecture document lists the same object as model id, endpoint, parameters, prompt hashes, split, workflow and run id. The tools read cfg["model"]["id"], cfg["workflow"] and cfg["split"] from it. The prompt hash observed for all runs has the prefix 70e815de3c01. Then one line per attempt: record id, scenario id, workflow, run id, raw text, parsed object or null, error or null, retries, latency, timestamp. A record may appear more than once if a run was resumed; the last line is authoritative at scoring. File names replace the colon in the Ollama identifier with an underscore (llama3.1_8b), which is why tools/admin_amount.py locates the raw file with mid.replace(':', '_').

5.4 Scored row (results/scored_test_<workflow>_<model>_<run>.jsonl). The first line is again the _config object copied from the raw file; then one row per record. Fields and their meaning:

Field	Meaning
record_id, scenario_id, workflow, run_id	Identity of the row, joined from the raw line and the scenario
age_band, complexity, abstention_case, conflict flag, challenges, generation_method	Scenario and record attributes carried through for stratification; abstention_case is the flag the tools test
malformed	True when the last attempt's parsed object is null; such a row scores incorrect on all fields
fields	A dictionary of per-field correctness over the required fields; REQUIRED = ["drug", "current_weight_kg", "dose_basis", "weight_normalized_dose", "dose_unit", "dose_per_admin_mg", "frequency_per_day", "route"], with dose_per_admin_mg dropped when the scenario carries the derive_per_dose challenge
field_acc	Mean of the values in fields (Level 1)
complete_regimen	True when every value in fields is True (Level 2)
codes	List of automatic error classes assigned by auto_code, plus arithmetic_inconsistency appended at Level 4 when applicable
obs_mg_per_kg_per_dose, obs_mg_per_kg_per_day	The observed exposure: in workflow B the model's reported mg_per_kg_per_dose and mg_per_kg_per_day; in workflow C the calculator's reconstruction
dev_dose, dev_day	Relative deviation of each observed value from the scenario's expected value; None when the observed value is missing
dosing_consistent	True when the worse of the available deviations is at most tol (0.10) (Level 3)
dose_consequential	True when the worse deviation exceeds tol or when no deviation is available (Level 4 composite)
dose_consequential_sens	The same test at tol_sens (0.20)
exceeds_labeled_max	True when the scenario has max_per_day_mg, the observed daily exposure is present, and that exposure times the current weight in kg exceeds max_per_day_mg times 1.001; per-day maxima only
primary_endpoint	complete_regimen and dosing_consistent
appropriate_abstention	Abstention cases only: every expected missing field is named in insufficient_information and left null
unnecessary_abstention	Abstention cases only: the model flagged a field that was not expected to be missing
hallucinated	Abstention cases only: the model supplied a value for an expected missing field

For abstention cases the scorer sets dosing_consistent and dose_consequential to None and returns before the Level 3 and 4 block, so those rows carry no observed exposure fields.

5.5 Manual codes (results/manual_codes.jsonl, results/manual_codes_recode.jsonl). One line per coded residual error: source file, record id, note text, parsed output, automatic codes, manual code; the re-code file adds the second code. A discarded assisted pass is retained as results/manual_codes_assisted_pass_discarded.jsonl.

5.6 Supplementary CSVs (added after registration). Columns are those written by the tools' csv.DictWriter calls.

results/admin_amount_run1.csv, one row per non-abstention run1 record per model and workflow: model, workflow, record_id, scenario_id, complexity, drug, reference_mg, written_mg, relative_deviation, status, dose_consequential_level4. status takes the values no_reference, missing, off or within; relative_deviation is rounded to four decimals and left empty when no comparison was made; dose_consequential_level4 copies the scored row's dose_consequential so the written amount can be cross-tabulated against the composite.

results/by_stratum_run1.csv, long format, one row per stratum, model and workflow: stratum_kind, stratum, model, workflow, n, primary_endpoint_rate, composite_failure_rate. stratum_kind takes the values all, complexity, age_band, generation, challenge and drug; rates are rounded to three decimals.

results/per_drug_run1.csv, one row per model, workflow and drug: model, workflow, drug, n_non_abstention, primary_endpoint_rate, dose_consequential_rate, n_abstention, appropriate_abstention. The study report records 160 rows for the four completed models.

results/max_check_run1.csv: written by tools/max_check.py over the 2,432 non-abstention run1 outputs of the four completed models in both workflows. The study report states that it flags an output when dose_per_admin_mg exceeds max_per_dose_mg, or dose_per_admin_mg times frequency_per_day exceeds max_per_day_mg, each with a 0.1 percent tolerance, using the model's own extracted maxima, and that a parallel column uses the scenario's maxima. The exact column names are not recorded in the sources used here.

5.7 Analysis text outputs. results/analysis_primary.txt (locked analysis on run1), results/analysis_primary_locked.txt and results/analysis_primary_corrected.txt (the comparison of Section 7), results/analysis_primary_four_models.txt, results/analysis_stability.txt and results/analysis_stability_four_models.txt, results/analysis_pairwise.txt and results/analysis_pairwise_four_models.txt, results/per_drug_run1.txt, results/admin_amount_run1.txt, results/by_stratum_run1.txt. results/export/ holds summary.json (every quoted number), tables as CSV, the run manifest, LOCK.json, DEVIATIONS.md and the sanitised configuration.

6. Generation pipeline

6.1 Sources. sources/fetch_labels.py pulls one label per drug from openFDA, with query overrides for salts such as amoxicillin and clavulanate potassium. The author verified each row of sources/dosing_sources.csv against the label text using tools/show_dosing.py and tools/set_row.py; the notes column records which maxima are stated by the label and which are the author's derivations. sources/check_sources.py enforces completeness and flags a drug carrying both per-dose and per-day rows; gentamicin carries both, which was accepted and logged. The table has 22 verified rows for 21 drugs: amoxicillin, amoxicillin-clavulanate, cephalexin, cefdinir, azithromycin, clindamycin, fluconazole, ampicillin, ceftriaxone, gentamicin (per-dose and per-day regimens), acetaminophen IV, ibuprofen, ondansetron, dexamethasone, prednisolone, famotidine, levothyroxine, digoxin, fentanyl, midazolam, vancomycin. The provenance of the maxima differs by drug and is recorded in the notes column: the ceftriaxone 2000 mg per day maximum is stated by the label; the amoxicillin 875 mg per dose ceiling is the label's adult regimen dose applied as a per-dose ceiling, the author's derivation; the prednisolone 60 mg per day figure is the upper end of the label's general initial-dose range applied as a ceiling, the author's derivation.

6.2 Growth references and age bands. Weight is sampled from the WHO weight-for-age tables for 0 to 24 months (boys and girls files merged by hand into sources/wtageinf.csv, 50 rows) and the CDC weight-for-age tables for 2 to 20 years (sources/wtage.csv), both as LMS tables. build_corpus.py samples age within band, sex, and weight from the LMS tables with z in minus 2 to plus 2, 10 percent tails at z 2.05 to 2.5, and a cap at 2.2 times the median. Age bands (sources/age_bands.json): neonate 0 to 27 days; infant 28 to 365 days; toddler and young child 366 to 2190 days, which collapses the NICHD categories toddler and early childhood; school-age child 2191 to 4380 days (NICHD middle childhood); adolescent 4381 to 6574 days.

6.3 Templates and complexity. generator/templates.py renders a note at the requested complexity level with a frequency vocabulary. Level 1: a bare order with weight, drug, dose, frequency and route. Level 2: an HPI and plan with vitals, a second medication and a computed per-dose amount with volume. Level 3: an admission note adding a prior-visit weight with its date, a discontinued order for the same drug, and in some records weights in pounds. build_corpus.py chooses a drug permitted for the age, draws a dose within the label range, and renders the note.

6.4 Distractors. A second medication (cholecalciferol under 6 months, cetirizine otherwise); a prior-visit weight dated relative to age so it is never before birth; a discontinued order for the same drug; pounds in some Level 3 notes; computed per-dose amounts and volumes in Level 2 and Level 3 notes, withheld in 36 Level 3 scenarios. Of the 103 non-abstention Level 3 scenarios, 36 withhold the per-dose amount (challenge tag derive_per_dose) and 67 print it.

6.5 Abstention cases. 57 records in which a required element is absent ("interval to be confirmed") or only a dated prior weight is present. The correct response is null with the field named in insufficient_information.

6.6 Paraphrase and audit. A fraction of records is paraphrased by qwen2.5:14b and audited by the same model with a committed rubric (generator/audit.py); flagged records are reviewed by the author. On the test set the first automated audit flagged three generator defects (unbounded tail weights to 108 kg; prior-visit dates before birth; cetirizine in young infants), all fixed and the set rebuilt; the second audit flagged 45 of 363 records, all reviewed, with no generator error found. Two Level 3 neonatal records with pre-birth prior dates were removed and the generator corrected, giving 361. A blinded author self-audit of 36 records (tools/self_audit.py) found no note requiring correction and three reader entry slips, which are reported. The 108-record stratified repeat subset is written by tools/make_repeat_subset.py.

6.7 Reference definition of the printed per-dose amount. The note prints the per-dose amount as an order would, with practical rounding and with labelled maxima applied; the scenario's expected_dose_per_admin_mg is that printed amount. Level 2 scores extraction of the printed amount at a 1 percent tolerance. Levels 3 and 4 compare mg/kg/dose and mg/kg/day with the canonical regimen, not with the printed amount. An audit of the 304 non-abstention scenarios (caps applied, units matched) found that about 53 print an amount more than 1 percent from the exact product and one (neonatal dexamethasone, 0.02 mg printed for an exact 0.0226 mg, a 13 percent gap) is more than 10 percent from it. Consequences for scoring: (a) a model that reports the exact product rather than the printed amount is scored incomplete at Level 2 and correct at Levels 3 and 4, which happened to GPT-5.5 in nine workflow B records; (b) on the one neonatal record no single internally consistent per-dose amount satisfies both the 1 percent Level 2 tolerance on the printed amount (0.0198 to 0.0202 mg) and the 10 percent Level 4 tolerance on the mg/kg figure read back from it (equivalent to 0.0203 to 0.0249 mg). The scorer is locked and applied as registered; the record is disclosed, and the pre-registered 20 percent sensitivity removes it. A Level 2 sensitivity against the exact product is listed as exploratory follow-up. The rounding audit list (the 53 records and the one above 10 percent) is an item still to be added to the evidence packet.

7. Lock and integrity

7.1 Lock procedure. tools/lock_testset.py writes data/test/LOCK.json with the record count, the SHA-256 of records.jsonl, scenarios.jsonl and repeat_subset.txt as raw bytes, and scoring_sha256 computed by hash_scoring() as one SHA-256 over the concatenated bytes of seven files. The lock was written on 3 October

2026 (commit 528c2b5) after the audits and the self-audit and before Protocol v1.0 was registered on OSF; no evaluated model saw a test record before the lock. The two hash functions, verbatim from tools/lock_testset.py:

```
    def hash_file(p):

        h = hashlib.sha256(); h.update(open(p, "rb").read()); return h.hexdigest()

    def hash_scoring():

        h = hashlib.sha256()

        for p in ("scoring/score.py", "generator/calculator.py", "harness/prompts/system.txt",
"harness/prompts/task_B.txt", "harness/prompts/task_C.txt", "harness/output_schema.json",
"analysis/analyze.py"):

            h.update(open(os.path.join(ROOT, p), "rb").read())

        return h.hexdigest()
```

7.2 The seven hashed files. scoring/score.py, generator/calculator.py, harness/prompts/system.txt, harness/prompts/task_B.txt, harness/prompts/task_C.txt, harness/output_schema.json and analysis/analyze.py. The scoring hash is one digest over all seven concatenated in that order; it is not the SHA-256 of score.py alone, a point the study report corrected in an earlier draft and logged as a clarification on 5 October.

7.3 Lock values. LOCK.json content: locked_at 2026-10-03; records_sha256 492d7ca2209719eb8bf96548403dab1e26cb9fed36be3482733dd69790cea7f4; scenarios_sha256 432be74fe65c2164ad9e21705821f5814055994f389526614aa744a671c220cb; repeat_subset_sha256 f1012bdcb01f9cdcf80cf000b0ad6831352d9a98a6e57a0e8ebd207dbdfa4196; scoring_sha256 7be9bc8704e09b94014741847f16b047b115c7b2c7c7846ec01afe03445461ed; n_records 361; smoke false. The same hashes are in protocol/LOCK_hashes.txt and in the OSF registration (osf.io/3p9ey, DOI 10.17605/OSF.IO/3P9EY).

7.4 How the scorer checks the lock. scoring/score.py check_lock(split) loads data/test/LOCK.json and exits if hash_file(records.jsonl) or hash_scoring() differs from the stored values; this runs on every scoring invocation. harness/runner.py refuses to start a test run if data/test/LOCK.json is absent. Together these give two refusals: a run cannot begin on an unlocked split, and a scoring pass cannot complete on a changed scorer, calculator, prompt, schema, registered analysis or record file.

7.5 Verification commands. Three checks reproduce the lock: shasum -a 256 on the three data files; recomputation of hash_scoring() in Python; and a git log over the seven hashed files from the lock commit to HEAD, which must list no commits. The commands are given verbatim in the runbook (Section 14). Verification on 5 October 2026 found the three data files recompute to the registered values; scoring/score.py was last changed in commit 7c948c4 (3 October, before the lock) and git reports the working file identical to that commit (blob ace4c0fa); no commit after the lock commit touched any of the seven files.

7.6 The 5 October discrepancy and its resolution. The first recomputation of hash_scoring() on 5 October returned a different value from the registered one. The cause was that the malformed-row correction (Section 10, analyze_malformed_as_failure.py) had been applied in the working copy of analysis/analyze.py, one of the hashed files. The locked analyze.py was restored byte for byte from the repository; the correction was moved to analysis/analyze_malformed_as_failure.py outside the hashed set, with its diff saved as

results/logs/analyze_malformed_as_failure.diff (15 lines); hash_scoring() then returned the registered value 7be9bc8704e09b94014741847f16b047b115c7b2c7c7846ec01afe03445461ed. The locked and corrected scripts produce identical run1 tables (results/analysis_primary_locked.txt and results/analysis_primary_corrected.txt differ only in the print order of one line). All scoring to date had been performed before the working-copy change, under the registered scorer; the scorer's refusal rule is what surfaced the discrepancy. The evidence packet carries the commands and their output (01_lock_verification.txt).

7.7 Rule for changes after lock. Any change that affects a result is a dated row in protocol/DEVIATIONS.md with its rationale. A change to any of the seven hashed files would break the scorer's check and is not made; a corrected analysis is instead implemented as a new script outside the hashed set, its diff is retained, and the deviation row records both the correction and where it lives. The deviation rows to date, from the study report: Workflow A (rules baseline) not run; resumed runs re-query records with no parsed output and the scorer keeps the last attempt; Gemini requests split across two Google Cloud projects; two DeepSeek run2 records persistently empty and scored malformed; H3 wording (Section 1 versus Section 16); the age_bands.json verification note replaced by the band mapping statement; the malformed-row correction and its relocation; "incorrect inference" folded into unsupported-value hallucination; pairwise Holm-corrected contrasts implemented after registration and the mixed-effects regression not performed; the assisted coding pass discarded; the clarification that dose_consequential is a composite and that exceeds_labeled_max tests per-day maxima only; the neonatal rounding record; the written per-dose amount analysis; the stratified table and heat maps; H3 reported under both wordings; the clarification that workflow B Level 4 scores the model's reported mg/kg values; the clarification of the scoring hash's scope; the post hoc maximum check; the neonatal probe; and model identifiers described as used with timestamps and the local digest.

7.8 Integrity controls, as tabulated in the architecture document.

Control	Mechanism
Test set immutability	SHA-256 in LOCK.json, published on OSF; runner and scorer refuse on mismatch
Scorer, calculator, prompt, schema and registered-analysis immutability	One combined hash at lock; the scorer recomputes it on every run; any change requires a logged deviation; a working-copy edit to the analysis script on 5 October was caught by this check, reverted, and moved outside the hashed set
Model pinning	Exact identifiers in config/models.yaml and in each raw file's _config line
Generator and evaluated model separation	Family check in harness/common.py refuses overlap
Resumability	Done set keyed on parsed output; last attempt wins at scoring; identical for all models
Blinding of manual coding	Model name hidden; randomised order; fixed seed; append-only file; unassisted
Secrets	.env git-ignored; secret_scan.py before any push, including results/logs; keys rotated after the study
Budget	200 USD ceiling in config; spend recorded per provider in the manifest
Deviations	protocol/DEVIATIONS.md, one dated row per change with rationale

8. Execution

The protocol registered three workflows: a rules-only Workflow A with no language model, which was not run (recorded in the deviations log, with the reason that it lies outside the hypotheses and that the calculator in the

hybrid workflow already provides the non-model arithmetic reference), and Workflows B and C, which keep their registered letters because the letters are fixed in the registration, in the locked prompt files and in every result file; renumbering them after registration would make the results disagree with the registered design.

8.1 Runner semantics. harness/runner.py takes split, workflow, run id, optional model and optional subset. For each model it reads the existing raw output file, marks records with a parsed object as done, and queries the rest; a record is marked done only when a parsed object exists, so a record whose last attempt is null is re-queried on restart. Each call uses the committed system prompt, the workflow task prompt and the record text. The response is parsed with extract_json; a non-JSON response triggers one repair message; a still-unparseable response after that second failure is stored with parsed null. Transient errors are retried; billing, credit, quota and rate-limit errors stop that model's loop with a message to rerun later. Because resumption appends new attempt lines rather than editing old ones, a record may appear more than once in a raw file, and the scorer keeps the last attempt. This gives unparsed outputs an extra attempt on a resumed run; the behaviour is identical for all models and is a logged deviation with before and after malformed counts reported. After resumption, malformed outputs were 0 of 361 for every completed model on run1.

8.2 Workflows. Workflow B asks the model to extract all fields, to compute dose_per_admin_mg, and to report mg_per_kg_per_dose and mg_per_kg_per_day. Workflow C asks for the fields only; the deterministic calculator computes exposure at scoring time. The prompts (harness/prompts/system.txt, task_B.txt, task_C.txt) and the output object (harness/output_schema.json) are locked. The committed system prompt specifies the output object with an example (amoxicillin values), the abstention rule, and no rule on the permissible age of a documented weight; the absence of a staleness rule is a reported finding rather than a defect to be fixed under the lock. Two prompt defects were found and fixed on development data before the lock: no output schema (models invented key names) and JSON-Schema type descriptors copied literally by one model; the final prompt shows a concrete example object. The prompt hash written to every raw file's _config line has the prefix 70e815de3c01.

8.3 Per-provider quota and credit behaviour. Gemini 3.1 Pro preview: 250 requests per model per Google Cloud project per day on the paid tier; worked around with a second project under the same billing account and a key swap in .env (both keys retained, one active; logged as a deviation). OpenAI and Anthropic: prepaid credit exhaustion stops the runner with a 429 or 400; top up and rerun the same command. DeepSeek: 402 on insufficient balance; two repeat-run records returned empty responses on every attempt (three attempts) and are scored malformed. On 4 October 2026, Gemini workflow C run1 completed (361) and workflow B run1 reached 342 of 361 before the per-project quota stopped it; completion of the remainder plus the repeat runs in both workflows, about 450 calls split across the two projects, was planned for the next quota reset.

8.4 Running and resuming. One background job per chain (caffeinate plus nohup); progress is read from the output files by counting records with parsed output; any interrupted command is rerun unchanged to resume. The Gemini chain runs workflow B run1 and then the four repeat runs (C and B, run2 and run3) on the subset:

```
    PYTHONUNBUFFERED=1 caffeinate -i nohup zsh -c 'python -m harness.runner --split test --workflow
B --run-id run1 --model gemini-3.1-pro-preview; for w in C B; do for r in run2 run3; do python -m
harness.runner --split test --workflow $w --run-id $r --subset data/test/repeat_subset.txt --model
gemini-3.1-pro-preview; done; done' > results/logs/gemini_evening1.log 2>&1 &

    sleep 60; tail -n 3 results/logs/gemini_evening1.log
```

8.5 The Gemini two-project procedure. Both Google keys are kept in .env under GOOGLE_API_KEY and GOOGLE_API_KEY_PROJECT1; the active key is the one under GOOGLE_API_KEY. When the first project's daily quota is exhausted, the two values are exchanged and the chain is rerun unchanged, so the runner resumes on the records without parsed output using the second project's quota. The exact key-swap block:

```
    python3 - <<'PY'

    L=open(".env").read().splitlines(); d={l.split("=",1)[0]:i for i,l in enumerate(L) if "=" in l}

    a,b=d["GOOGLE_API_KEY"],d["GOOGLE_API_KEY_PROJECT1"]; va=L[a].split("=",1)[1];
vb=L[b].split("=",1)[1]

    L[a]="GOOGLE_API_KEY="+vb; L[b]="GOOGLE_API_KEY_PROJECT1="+va; open(".env","w").write("\
n".join(L)+"\n"); print("swapped; active key now ends with",vb[-4:])

    PY
```

The same model identifier, endpoint and parameters are used under both projects, which is what the deviation row records.

8.6 The status block. Progress is read from the raw files, not from the log: the block counts, per raw file, the distinct record ids with a parsed object, prints the count against the target (361 for run1 files, 108 for repeat files) and marks DONE when the target is reached; the second line reports whether a runner process is alive.

```
    python3 - <<'PY'

    import json,glob

    for f in sorted(glob.glob("results/raw_test_*_run1.jsonl"))+sorted(glob.glob("results/
raw_test_*_run[23].jsonl")):

        ok=set()

        for l in open(f):

            o=json.loads(l)

            if "record_id" in o and o.get("parsed") is not None: ok.add(o["record_id"])

        target=361 if "run1" in f else 108

        print(f"{len(ok):4d} / {target}  {'DONE' if len(ok)>=target else '':5s} {f.split('/')[-1]}")

    PY

    pgrep -f harness.runner >/dev/null && echo "runner: RUNNING" || echo "runner: STOPPED"
```

8.7 Standard sequence. SETUP_GUIDE.md BLOCKS 0 to 8: 0 environment and keys; 1 tests and smoke; 2 sources; 3 development corpus and runs; 4 test corpus, audits, self-audit, lock; 5 protocol finalisation and OSF registration; 6 test runs (run1 both workflows, repeats run2 and run3 on the subset); 7 scoring, analysis, stability, manual coding; 8 export and hand-off. The full study is 30 run files: five models, two workflows, three runs each.

9. Scoring rules in full

scoring/score.py reads each raw file, keeps the last attempt per record, joins the scenario, normalises units, compares fields, scores abstention cases and assigns automatic error classes by rules. The rules below are as implemented in the locked file; the Level 3 and 4 path is quoted verbatim.

9.1 Level 1, field accuracy. REQUIRED = ["drug", "current_weight_kg", "dose_basis", "weight_normalized_dose", "dose_unit", "dose_per_admin_mg", "frequency_per_day", "route"]. Each required field is compared with the scenario target after normalisation by field_correct; numeric fields use NUM_TOL = 0.01, a 1 percent relative tolerance. The targets object t sets dose_per_admin_mg from the scenario's expected_dose_per_admin_mg, that is, the printed, rounded, maximum-applied amount. Non-scalar values are treated as missing (a scorer fix made before the lock). field_acc is the mean of per-field correctness over the required fields.

9.2 Level 2, complete regimen, with the withheld-amount rule. complete_regimen is True when every required field is correct. When the scenario carries the derive_per_dose challenge, meaning the note withholds the per-dose amount, dose_per_admin_mg is removed from the required list and does not enter Level 2; this applies to the 36 withholding Level 3 scenarios. In workflow C, where the model is asked to extract and not to compute, a missing amount on a withholding note is correct behaviour and is not a failure.

9.3 Level 3, dosing consistency. The observed mg/kg per dose and per day are compared with the scenario's expected_mg_per_kg_per_dose and expected_mg_per_kg_per_day by relative_deviation. dosing_consistent is True when the worse of the available deviations is at most tol (0.10). Where one observed value is missing, its deviation is None and is excluded from the maximum; where both are missing, worst is None and dosing_consistent is False.

9.4 Level 4, composite dose consequence, as implemented for B and for C. In workflow B the observed values are the model's own reported mg_per_kg_per_dose and mg_per_kg_per_day. In workflow C they are computed by generator/calculator.py through reconstruct from the model's extracted current_weight_kg, dose_basis, weight_normalized_dose, dose_unit, frequency_per_day, max_per_dose_mg, max_per_day_mg and concentration_mg_per_ml; any exception in the calculator leaves both observed values None. The written per-dose amount, dose_per_admin_mg, does not enter Level 4 in either workflow. The verbatim path:

```
    req = [k for k in REQUIRED if not (k == "dose_per_admin_mg" and "derive_per_dose" in
(s.get("challenges") or []))]

    fields = {k: field_correct(k, p.get(k), t[k]) for k in req}

    out["fields"] = fields

    out["field_acc"] = sum(fields.values()) / len(fields)

    out["complete_regimen"] = all(fields.values())

    out["codes"] = auto_code(p, s, t)

    if abst:

        flagged = set(p.get("insufficient_information") or [])

        expect = {"current_weight_kg" if f == "current_weight_value" else f for f in miss}
```

```
        out["appropriate_abstention"] = expect.issubset(flagged) and all(p.get(f) is None for f in
expect)

        out["unnecessary_abstention"] = bool(flagged - expect)

        out["hallucinated"] = any(p.get(f) is not None for f in expect)

        out["dosing_consistent"] = None; out["dose_consequential"] = None

        return out

    # Level 3/4

    if workflow == "C":

        try:

            rc = reconstruct(p.get("current_weight_kg"), p.get("dose_basis"),
p.get("weight_normalized_dose"), p.get("dose_unit"),

                             p.get("frequency_per_day"), p.get("max_per_dose_mg"),
p.get("max_per_day_mg"), p.get("concentration_mg_per_ml"))

            obs_dose, obs_day = rc["mg_per_kg_per_dose"], rc["mg_per_kg_per_day"]

        except Exception:

            obs_dose, obs_day = None, None

    else:

        obs_dose, obs_day = p.get("mg_per_kg_per_dose"), p.get("mg_per_kg_per_day")

    exp_dose, exp_day = s.get("expected_mg_per_kg_per_dose"), s.get("expected_mg_per_kg_per_day")

    dev_dose, dev_day = relative_deviation(obs_dose, exp_dose), relative_deviation(obs_day, exp_day)

    devs = [d for d in (dev_dose, dev_day) if d is not None]

    worst = max(devs) if devs else None

    out.update({"obs_mg_per_kg_per_dose": obs_dose, "obs_mg_per_kg_per_day": obs_day,

                "dev_dose": dev_dose, "dev_day": dev_day,

                "dosing_consistent": (worst is not None and worst <= tol),

                "dose_consequential": (worst is None or worst > tol),

                "dose_consequential_sens": (worst is None or worst > tol_sens),

                "exceeds_labeled_max": bool(s.get("max_per_day_mg") and obs_day is not None and
obs_day * to_kg(s["current_weight_value"], s["current_weight_unit"]) > s["max_per_day_mg"] *
1.001)})

    if out["complete_regimen"] and not out["dosing_consistent"]:
out["codes"].append("arithmetic_inconsistency")

    out["primary_endpoint"] = out["complete_regimen"] and out["dosing_consistent"]

    return out
```

tol is 0.10 and tol_sens is 0.20.

9.5 The composite and its two components. dose_consequential is True when the worse of the two relative deviations exceeds 10 percent (worst > tol) or when no exposure can be computed (worst is None, meaning both observed values are missing). The two components, "computed exposure more than 10 percent off" and "no exposure computable", are reported separately wherever the composite is reported, and the phrase "more than 10 percent off" is reserved for computed exposures. The 10 percent figure is adapted from a published modified-Delphi definition of pediatric dosing error (Roumeliotis et al., BMC Pediatrics 2020), whose panel treated doses more than 10 percent outside a reference range as errors for most intermittent medicines; the study applies it to a constructed intended dose as an operational threshold, not a harm claim, and the composite additionally counts an unreconstructable exposure, which the panel's definition does not.

9.6 The sensitivity flag. dose_consequential_sens applies the same rule at tol_sens, 20 percent, and is pre-registered. It removes the neonatal rounding record (9.11) from the composite count.

9.7 exceeds_labeled_max and its per-day-only scope. The flag is True when the scenario has max_per_day_mg, the observed daily exposure is present, and that exposure multiplied by the current weight in kilograms (to_kg over the scenario's current_weight_value and current_weight_unit) exceeds max_per_day_mg times 1.001. A per-dose maximum (amoxicillin, 875 mg) is not tested, which is why one of the six larger maximum-dose outputs carries the flag False. The flag does not feed the primary endpoint. In workflow C the calculator applies the cap, so the flag is False for the capped reconstruction even where the model's written amount was uncapped.

9.8 Primary endpoint. primary_endpoint is complete_regimen and dosing_consistent, on non-abstention records. A malformed output counts as a failure.

9.9 Abstention scoring. For an abstention case the expected set is the scenario's missing fields, with current_weight_value mapped to the output field current_weight_kg. appropriate_abstention is True when every expected field is named in insufficient_information and left null. unnecessary_abstention is True when the model flagged a field outside the expected set. hallucinated is True when the model supplied a value for any expected field. "Incorrect inference" is not scored as a separate category; it is folded into unsupported-value hallucination, a logged clarification since the scorer is locked. Abstention rows carry dosing_consistent and dose_consequential as None.

9.10 Automatic error classes. auto_code assigns classes by committed rules (Protocol Section 13): weight selection, weight unit, unit scale, dose basis, frequency, route, strength versus dose, dose cap, adult fixed-dose assumption, unsupported-value hallucination, conflict resolved without flag, arithmetic inconsistency, malformed output, extraction (value wrong, no finer class). Two rules deserve statement. The arithmetic_inconsistency rule is applied at Level 4, outside auto_code: when complete_regimen is True and dosing_consistent is False, "arithmetic_inconsistency" is appended to codes; this is how the six larger maximum-dose outputs of 9.12 and the Llama reported-field confusions were labelled. The dose_cap rule fires when the scenario has a maximum, the output reports none, and the per-dose amount is wrong or missing; seven of the 52 appropriate frontier abstentions were flagged dose cap by this rule because they also withheld a computable per-dose amount, which on inspection is a withheld computable field rather than a cap error. Residual extraction errors are coded manually (Section 10, code_errors.py) with classes E1 extraction, E2 normalisation, E3 association, E4 temporal, E5 volume versus dose, E6 other.

9.11 Malformed handling. A parsed null is a malformed output. It takes an early branch in the scorer with dose_consequential True and codes ["malformed_output"], and scores incorrect on all fields. The registered

analyze.py dropped malformed non-abstention rows from the bootstrap denominator; the 5 October correction counts them as failures on the primary endpoint, complete regimen and dosing consistency, in line with Protocol Section 12, and is implemented in analyze_malformed_as_failure.py. On run1 there are no malformed rows for the completed models and the two scripts agree; the only malformed rows to date are the two DeepSeek run2 records.

9.12 Two row-level cases the rules produce. Five non-abstention Level 3 notes give a weight-based rule, a maximum written into the order and no per-dose amount, and in each the weight-based product exceeds the maximum. Seven workflow B outputs (Claude Sonnet 5 and DeepSeek) wrote the uncapped product after placing the maximum in the correct structured field: six exceed the maximum by more than 10 percent and are composite failures labelled arithmetic_inconsistency; the seventh (Claude, prednisolone adolescent, 64.65 mg against 60 mg per day, 7.75 percent over) is inside the tolerance and passed Level 4 with dev_dose 0.0775, dose_consequential False and primary_endpoint True, while exceeds_labeled_max is True for it. The tolerance therefore passes a cap violation that the per-day flag records but the endpoint ignores. GPT-5.5 applied the maximum in all five notes.

9.13 The neonatal probe and the absence of a consistency check between fields. test-dexamethasone-mg_kg_day-neonate-L1-v0-r0: a 17-day-old, 4.52 kg, dexamethasone 0.02 mg/kg/day PO divided four times daily, note prints "0.02 mg per dose"; exact per-dose 0.0226 mg. Claude Sonnet 5 (B) wrote dose_per_admin_mg 0.02 and mg_per_kg_per_dose 0.0044; the scorer's observed mg/kg/dose is 0.0044 against 0.005 expected, dev_dose 0.12, dev_day 0.0, dosing_consistent False, dose_consequential True, dose_consequential_sens False, exceeds_labeled_max False. GPT-5.5 and DeepSeek (B) wrote 0.0226 mg: all fields correct except dose_per_admin_mg (13 percent from the printed 0.02 at a 1 percent tolerance), dev_dose 0.0, dose_consequential False, primary_endpoint False at Level 2. On 5 October a constructed output identical to Claude's but with dose_per_admin_mg 0.02, mg_per_kg_per_dose 0.005 and mg_per_kg_per_day 0.02 was written to a probe raw file and scored by the locked scorer: field accuracy 1.0, complete regimen True, dev_dose 0.0, dosing_consistent True, dose_consequential False, primary endpoint True. What this shows: workflow B scores dose_per_admin_mg at Level 2 against the printed amount and the model's reported mg/kg fields at Level 4 against the canonical regimen, with no consistency check between the two, so an output whose amount field and computed field disagree with each other by 13 percent passes both levels. No model produced such an output. An earlier draft's claim that no output could pass both levels was too strong and is withdrawn. The probe files were removed and the Claude scored file verified unchanged by hash; the deviation row records the probe.

10. Analyses

Each script is listed with its purpose, inputs, rules, outputs, usage line and what it found on the four completed models (GPT-5.5, Claude Sonnet 5, DeepSeek and Llama 3.1 8B, run1, n = 304 non-abstention records per model and workflow). Gemini rows are pending and are added from the same scripts when its runs complete.

10.1 analysis/analyze.py (locked). Purpose: the registered primary analysis. Inputs: the run1 scored files, glob 'results/scored_test_*_run1.jsonl'. Rules: per model and workflow it reports the primary endpoint, field accuracy, complete regimen and composite failure rate with clustered bootstrap confidence intervals (2,000 resamples over scenarios); abstention metrics; breakdowns by complexity, band and generation method; malformed counts; top error codes; and the H3 paired comparison on the composite rate with its interval and verdict. The bootstrap seed used by analyze.py is not recorded in the sources used here. Output: results/analysis_primary.txt (and the four-model file results/analysis_primary_four_models.txt). Usage:

```
    python -m analysis.analyze --scored 'results/scored_test_*_run1.jsonl' | tee
results/analysis_primary.txt > /dev/null
```

Findings. Primary endpoint and field accuracy:

Model	Workflow	Field accuracy	Complete regimen	Composite failure	Primary endpoint
gpt-5.5-2026-04-23	B	0.996	0.970 (0.947 to 0.987)	0.000	0.970 (0.947 to 0.987)
gpt-5.5-2026-04-23	C	1.000	1.000	0.000	1.000
claude-sonnet-5	B	1.000	0.997 (0.990 to 1.000)	0.016 (0.003 to 0.033)	0.980 (0.964 to 0.993)
claude-sonnet-5	C	1.000	0.997 (0.990 to 1.000)	0.000	0.997 (0.990 to 1.000)
deepseek-v4-pro	B	1.000	0.997 (0.990 to 1.000)	0.007 (0.000 to 0.016)	0.990 (0.977 to 1.000)
deepseek-v4-pro	C	1.000	1.000	0.000	1.000
gemini-3.1-pro-preview	B	pending	pending	pending	pending
gemini-3.1-pro-preview	C	pending	pending	pending	pending
llama3.1:8b	B	0.968	0.809 (0.766 to 0.855)	0.645 (0.592 to 0.697)	0.329 (0.276 to 0.382)
llama3.1:8b	C	0.759	0.062 (0.036 to 0.089)	0.891 (0.855 to 0.924)	0.056 (0.030 to 0.082)

Composite failure counts behind the rates: Claude B 5 (four maximum-dose outputs and the neonatal rounding record); DeepSeek B 2 (both maximum-dose outputs); GPT-5.5 both workflows, Claude C and DeepSeek C 0; Llama B 196, of which 189 reported exposures more than 10 percent off and 7 not computable; Llama C 271, of which 2 and 269. Abstention (n = 57): frontier models 52 of 57 appropriate (0.912, CI 0.842 to 0.982), identical across models and workflows because the same five records account for every miss; each of those notes documents only a weight dated at a prior visit (2025-12-01) against an order dated 2026-03-10, and every frontier model supplied that weight as current. Llama 0.772 (B) and 0.825 (C). Moderators: paraphrased notes scored lower than template notes in workflow B for every frontier model (Claude 0.948 versus 1.000; GPT-5.5 0.957 versus 0.979; DeepSeek 0.974 versus 1.000); all seven frontier composite failures occurred in paraphrased records. Error code counts: Llama C extraction dose wrong 270, adult fixed-dose assumption 261, frequency 25, arithmetic 2; Llama B arithmetic inconsistency 146, frequency 27, extraction dose wrong 17, dose basis 14, weight wrong 1; Claude B arithmetic 5, dose basis 1; Claude C dose basis 1; DeepSeek B arithmetic 2; DeepSeek C and GPT-5.5 none. GPT-5.5 workflow B: nine records fail only on dose_per_admin_mg with no error class and no composite failure, the model having reported the exact product (6.15 kg times 2 mg/kg is 12.3 mg) where the order printed a rounded amount (12 mg).

Hypothesis ledger (analysis version: analyze.py with the malformed-row correction; run1; four models):

Hypothesis	Pre-specified rule	Result	Verdict
H1	Field accuracy minus complete-regimen accuracy at least 0.10 for every model	GPT-5.5 0.026 (B), 0.000 (C); Claude 0.003, 0.003; DeepSeek 0.003, 0.000; Llama 0.159, 0.697	Not supported (holds for Llama only)
H2	Composite failure rate rises monotonically L1 to L3	Frontier near zero at all levels; Llama B 0.634, 0.700, 0.602; Llama C 0.851, 0.940, 0.883	Not supported
H3 (primary)	C minus B composite failure	GPT-5.5 +0.000 (null at ceiling);	Not supported: at most 2 of 5

	rate below zero with CI excluding zero, for a majority of models	Claude -0.016 (-0.033 to -0.003), supported; DeepSeek -0.007 (-0.016 to +0.000), not supported; Llama +0.247 (+0.178 to +0.312), reversed; Gemini pending	can meet the rule
H4	Dose-basis and weight-selection classes are the largest share of composite failures	Largest automatic classes: adult fixed-dose assumption and arithmetic inconsistency (Llama); dose basis distant third; weight selection rare	Not supported as automatically coded; manual coding will settle the class
H5 (exploratory)	Neonate and infant bands lower than adolescent	Frontier: no band effect; Llama B 0.571 adolescent, 0.638 infant, 1.000 neonate; Llama C 0.929 adolescent, 0.914 infant, 0.462 neonate	Inconclusive; neonate estimates oppose across workflows

H3 under both protocol wordings. Section 1 of Protocol v1.0 (verbatim): "H3. The hybrid workflow (C) will have a lower dose-consequential error rate than the end-to-end workflow (B) for the same model (primary comparison)." Section 16 (verbatim): "Primary comparison (H3): Workflow C minus Workflow B on the primary endpoint, per model, paired at the record level, with clustered bootstrap 95% CI. The comparison is declared supported if the CI excludes zero in the hypothesized direction for the majority of evaluated models." The two name different outcome measures. analyze.py implements H3 on the composite rate (Section 1); because it is one of the seven hashed files whose combined hash was published in the registration before any test run, the analysed definition was fixed before any result existed. The Section 16 difference on the primary endpoint is computed with pairwise.py's load and paired_boot functions. Section 16's rule (paired, clustered, majority) applies to either measure.

Model	C minus B, composite failure rate (Section 1)	C minus B, primary endpoint (Section 16)
gpt-5.5-2026-04-23	+0.000 (null at ceiling)	+0.030 (+0.013 to +0.049), supported; the entire gain is the nine Level 2 rounding records, no dose changed
claude-sonnet-5	-0.016 (-0.033 to -0.003), supported	+0.016 (+0.003 to +0.033), supported
deepseek-v4-pro	-0.007 (-0.016 to +0.000), not supported	+0.010 (+0.000 to +0.023), not supported
gemini-3.1-pro-preview	pending	pending
llama3.1:8b	+0.247 (+0.178 to +0.312), reversed	-0.273 (-0.336 to -0.211), reversed
Verdict	Under Section 1: not supported, at most 2 of 5	Under Section 16: two of four completed; supported only if Gemini also shows the effect (3 of 5)

10.2 analysis/analyze_malformed_as_failure.py. Purpose: the registered analysis with the malformed-row correction, in which malformed non-abstention rows count as failures on the primary endpoint, complete regimen and dosing consistency rather than being dropped from the bootstrap denominator. Inputs and rules otherwise identical to analyze.py; the diff against the locked file is 15 lines and is retained at results/logs/analyze_malformed_as_failure.diff. Output: results/analysis_primary_corrected.txt in the lock comparison. Usage line: not recorded in the sources used here; the script takes the same scored glob as analyze.py. Findings: identical output on run1, where no completed model has a malformed row; it is the script to use for any file containing malformed rows, at present the two DeepSeek run2 records.

10.3 analysis/stability.py. Purpose: run-to-run agreement on the 108-record repeat subset. Inputs: the three scored files (run1, run2, run3) of one model and one workflow. Rules: agreement across the three runs on the repeat subset, per field and per outcome. Output: appended per model and workflow to results/analysis_stability.txt by the loop in the runbook (results/analysis_stability_four_models.txt for the four completed models). Usage, as the loop over all five models and both workflows:

```
    rm -f results/analysis_stability.txt; for m in gpt-5.5-2026-04-23 claude-sonnet-5 gemini-3.1-pro-preview deepseek-v4-pro llama3.1_8b; do for w in C B; do echo "## $m wf=$w" >> results/analysis_stability.txt; python -m analysis.stability --scored results/scored_test_${w}_${m}_run1.jsonl results/scored_test_${w}_${m}_run2.jsonl results/scored_test_${w}_${m}_run3.jsonl >> results/analysis_stability.txt 2>&1; done; done
```

Findings: agreement 0.972 to 1.000 for all four completed models in both workflows. Llama at 1.000 reflects deterministic local inference at the chosen settings, not robustness, so its figure is not comparable to the API models. Lowest frontier values: Claude C complete regimen 0.972; GPT-5.5 B dose per administration 0.981; DeepSeek B 0.991 on all fields. Two DeepSeek B run2 records returned empty responses on three attempts and are malformed.

10.4 analysis/pairwise.py (added after registration). Docstring, verbatim:

```
"""Secondary pairwise model contrasts on the primary endpoint (Protocol Section 16).



For each workflow and each pair of models, the difference in complete dose-consistent regimen accuracy

on non-abstention run1 records, paired at the record level, with a clustered bootstrap (resampling

scenarios, 2,000 draws) for the 95 percent interval and a two-sided bootstrap p value. Holm correction

is applied across all contrasts. Malformed outputs count as failures (Section 12).



Usage: python -m analysis.pairwise --scored 'results/scored_test_*_run1.jsonl'

"""
```

Inputs: the run1 scored files; options --B (default 2000) and --seed (default 1). Rules, from the code. load reads a scored file, takes the model id and workflow from the _config line, skips abstention rows, maps primary_endpoint to 1.0 or 0.0, treats a malformed row with no endpoint as 0.0, and drops any other row without an endpoint. paired_boot takes two such tables, restricts to the record ids present in both, forms the per-record difference a minus b and groups the differences by scenario id; the estimate is the mean of all differences; the bootstrap draws, with random.Random(seed), B samples of scenarios with replacement from the sorted scenario list, each of the same size as the number of scenarios, and takes the mean of the pooled differences in each sample; the interval is the percentile interval, the sorted bootstrap means at position int(0.025 * B) and at the smaller of B minus 1 and int(0.975 * B); the two-sided bootstrap p is min(1.0, 2 * min(le, ge)), where le is the fraction of bootstrap means at or below zero and ge the fraction at or above zero. holm sorts the raw p values, multiplies each by (m minus rank), caps at 1.0, and enforces monotonicity by a running maximum. main loads every file, groups by workflow, computes every pair of models within a workflow with

itertools.combinations, applies Holm across all contrasts in both workflows together, and prints one line per contrast with diff, interval, n, p, p_holm and a significance flag at 0.05. Output: results/analysis_pairwise.txt (results/analysis_pairwise_four_models.txt for the four completed models). Usage:

```
    python -m analysis.pairwise --scored 'results/scored_test_*_run1.jsonl' | tee
results/analysis_pairwise.txt
```

Findings: no statistically significant pairwise difference among the frontier models was detected in either workflow, which is not evidence of equivalence (Claude minus DeepSeek B -0.010, adjusted p 0.58; Claude minus GPT-5.5 B +0.010, adjusted p 1.00; DeepSeek minus GPT-5.5 B +0.020, adjusted p 0.46; all C contrasts within 0.003, adjusted p 1.00). Every frontier model exceeds Llama: B +0.641 to +0.661; C +0.941 to +0.944; all adjusted p below 0.0001. The mixed-effects regression named in the protocol as exploratory was not performed because it is degenerate at ceiling; this and the addition of pairwise.py are logged deviations, the analysis code outside the lock being changeable without breaking the scorer's check.

10.5 tools/per_drug.py (added after registration). Docstring, verbatim:

```
"""Per-drug breakdown of run1 results (supplementary table).


For each model and workflow: records per drug, primary endpoint rate, dose-consequential error rate,

and the count of abstention records. Writes results/per_drug_run1.csv and prints a compact table.

Malformed outputs count as failures (Section 12).


Usage: python -m tools.per_drug --scored 'results/scored_test_*_run1.jsonl'

"""
```

Inputs: the run1 scored files and data/<split>/scenarios.jsonl, from which the drug of each scenario is read. Rules: for each (model, workflow, drug) key, abstention rows increment n_abstention and, when appropriate_abstention is True, appropriate_abstention; non-abstention rows increment n_non_abstention, primary (with a malformed row counted as False) and dc (dose_consequential). Rates are rounded to three decimals and left empty when n is zero. Output: results/per_drug_run1.csv with the columns in 5.6, plus a printed compact table; the --out option changes the CSV path. Usage, as run:

```
    python -m tools.per_drug --scored 'results/scored_test_*_run1.jsonl' > results/per_drug_run1.txt
```

Findings: 160 rows for the four completed models. Frontier drug cells are at ceiling except amoxicillin, ceftriaxone, prednisolone and dexamethasone, which carry the maximum-dose and neonatal rounding records. Llama does best on the per-dose fixed-frequency drugs (azithromycin, famotidine, ondansetron, midazolam).

10.6 tools/admin_amount.py (added after registration; exploratory). Docstring, verbatim:

```
"""Exploratory secondary analysis (added after registration): the written per-dose amount.
```

```
For every model and workflow, on the 304 non-abstention run1 records, compares the per-dose amount
the

model actually wrote (dose_per_admin_mg in its parsed output) with the scenario's
expected_dose_per_admin_mg,

which is the printed order amount: practical rounding applied, labelled maximum applied, in mg. This
is a

different question from Level 4, which recomputes exposure from the extracted weight and mg/kg
value: here

the question is whether the number the model wrote down is itself usable.



Rules: a written amount is "off" when its absolute relative deviation from the reference exceeds 10
percent;

"missing" when the field is absent, null or non-numeric; records whose scenario has no reference
amount are

counted separately and excluded from the comparison. No unit conversion is applied: both sides are
mg.



Writes results/admin_amount_run1.csv (one row per record) and prints a table by model, workflow and
level.



Usage: python -m tools.admin_amount --scored 'results/scored_test_*_run1.jsonl'

"""
```

Inputs: the run1 scored files, the scenarios file of the split, and the matching raw file (results/raw_<split>_<workflow>_<model>_run1.jsonl with the colon in the model id replaced by an underscore, falling back to the scored path with "scored_" replaced by "raw_"), from which the last parsed object per record is taken. Rules, from the code: the reference is float(expected_dose_per_admin_mg); a reference of None or 0 gives status no_reference; a written value that is absent or not convertible to float gives status missing; otherwise dev is the absolute value of val minus ref, divided by ref, and status is off when dev exceeds --tol (default 0.10) and within otherwise. Counts are tabulated by model, workflow and complexity level and printed with totals. Output: results/admin_amount_run1.csv (columns in 5.6) and the printed table captured to results/admin_amount_run1.txt. Usage, as run:

```
    python -m tools.admin_amount --scored 'results/scored_test_*_run1.jsonl' | tee
results/admin_amount_run1.txt
```

Findings:

Model	Workflow	Compared	Off (more than 10 percent)	Missing, amount withheld in note	Missing, amount printed
gpt-5.5-2026-04-23	B	304	1 (neonatal rounding record)	0	0
gpt-5.5-2026-04-23	C	268	0	36	0

claude-sonnet-5	B	304	4 (the four maximum-dose outputs)	0	0
claude-sonnet-5	C	296	3 (amoxicillin, ceftriaxone, prednisolone school-age: uncapped amounts written in the extraction-only workflow)	8	0
deepseek-v4-pro	B	304	3 (two maximum-dose outputs; neonatal rounding record)	0	0
deepseek-v4-pro	C	277	0	27	0
llama3.1:8b	B	277	27 (Level 1: 2; Level 2: 1; Level 3: 24)	13	14
llama3.1:8b	C	270	21 (all Level 3)	16	18

Reading. Workflow B written-amount deviations among the frontier models number eight (GPT-5.5 1, Claude 4, DeepSeek 3), against seven composite failures (Claude 5, DeepSeek 2): the six larger maximum-dose outputs are in both counts; the exact-product neonatal amounts of GPT-5.5 and DeepSeek are written-amount deviations only; Claude's copied neonatal amount is a composite failure only; Claude's 64.65 mg output is in neither count, being 7.75 percent over. In workflow C a missing amount on a withholding note is correct behaviour, so GPT-5.5's 36 and DeepSeek's 27 are not failures; Claude computed the amount in 28 of the 36 withholding notes although not asked to, and three of those exceed the stated maximum by more than 10 percent, invisible to workflow C's Level 4, which recomputes exposure from the mg/kg field. For the small model, 14 of 27 missing amounts in B and 18 of 34 in C are on notes that print the amount. The small model's written amount was within 10 percent of the order in 250 (B) and 249 (C) of 304, against composite failures of 196 and 271; the reconciliation cross-tab (written-amount status against the composite and its components, from the dose_consequential_level4 column) is:

Workflow	Written amount	Composite pass	Reported or computed exposure more than 10 percent off	No exposure computable
B	within 10 percent (250)	96	139	15
B	off (27)	4	21	2
B	missing (27)	8	0	19
C	within 10 percent (249)	14	2	233
C	off (21)	0	0	21
C	missing (34)	19	0	15

Of the 154 Llama B records with a correct written amount and a failed composite, 74 carry the milligram amount in the per-dose mg/kg field, 48 the per-day value in the per-dose field, 15 a null, 17 other. In C, 233 records have a correct written amount and no computable exposure because the model labelled the order "fixed" and nulled the mg/kg value; in a typical trace the note states "28 mg/kg/day … (266 mg per dose)", the model returns dose_basis "fixed", weight_normalised_dose null, dose_per_admin_mg 266, and the calculator cannot form an exposure from a null weight-normalised dose. The earlier working figure of 28 off for Llama B was computed against the exact product; the table above uses the printed reference and supersedes it.

10.7 tools/by_stratum.py (added after registration; exploratory beyond the pre-specified strata). Docstring, verbatim:

```
"""Stratified results for the heat-map figures (supplementary; exploratory beyond the pre-specified
strata).



For each model and workflow, on non-abstention run1 records, the primary endpoint rate and the
composite

dose-consequential failure rate within each stratum: complexity level, age band, generation method,
each

challenge tag, and drug. Malformed outputs count as failures (Section 12). Strata are overlapping
for

challenge tags (a record carries several), so those rows do not sum to the total.



Writes results/by_stratum_run1.csv (long format) and prints two pivots (primary endpoint; composite
failure).



Usage: python -m tools.by_stratum --scored 'results/scored_test_*_run1.jsonl'

"""
```

Inputs: the run1 scored files and the scenarios file for the drug of each scenario. Rules, from the code: for each non-abstention row, pe is primary_endpoint (False when None and malformed) and dc is dose_consequential (True when None and malformed); the row is counted under the strata ("all", "all records"), ("complexity", "Level <n>"), ("age_band", band), ("generation", "paraphrase" when generation_method starts with "paraphrase", otherwise "template"), ("drug", drug) and one ("challenge", tag) stratum per challenge tag. Columns are ordered by the constant ORDER = ["gpt-5.5-2026-04-23", "claude-sonnet-5", "gemini-3.1-pro-preview", "deepseek-v4-pro", "llama3.1:8b"] and then workflow. Output: results/by_stratum_run1.csv (columns in 5.6) and two printed pivots, primary endpoint rate and composite failure rate, captured to results/by_stratum_run1.txt. Usage, as run:

```
    python -m tools.by_stratum --scored 'results/scored_test_*_run1.jsonl' | tee
results/by_stratum_run1.txt
```

Findings. Frontier models in workflow B: every non-zero cell is accounted for by the seven records of 9.12 and 9.13; the highest cell is "per-dose amount withheld" (Claude 0.111, 4 of 36; DeepSeek 0.056, 2 of 36), where the four maximum-dose notes sit; Claude's neonate cell (0.077) is the single rounding record among 13. Frontier models in workflow C: zero in every stratum. Llama B fails every per-day-basis order at 0.865 against 0.126 primary-endpoint success, fails all 13 neonatal and all 19 microgram-drug records, and does best on the per-dose fixed-frequency drugs. Llama C: 0.80 to 0.96 in every stratum except neonate (0.462) and microgram drugs (0.158), where the calculator recovers what the model's own arithmetic lost; these are the only strata in which the calculator helped the small model, and they are small (13 and 19 records). Complexity and age band were pre-specified moderators; the rest were not, and the table is reported as exploratory.

10.8 tools/max_check.py (added after external review; exploratory, post hoc). Purpose: would a check of the written per-dose amount against the maximum the model itself extracted have flagged the uncapped outputs. Inputs: the parsed outputs and scenarios for the non-abstention run1 records of both workflows (2,432 outputs for the four completed models). Rules: flag if dose_per_admin_mg exceeds max_per_dose_mg, or dose_per_admin_mg times frequency_per_day exceeds max_per_day_mg, each with a 0.1 percent tolerance, using the model's own extracted maxima; a parallel column uses the scenario's maxima. Output: results/max_check_run1.csv. Usage:

```
python -m tools.max_check
```

Findings:

Model	Workflow	Flagged by the model's own maxima	Flagged by scenario maxima
gpt-5.5-2026-04-23	B and C	0	0
claude-sonnet-5	B	5 (the four composite-failure outputs and the 64.65 mg output)	5
claude-sonnet-5	C	4 (the same notes except ceftriaxone replaced by the v0 prednisolone note)	4
deepseek-v4-pro	B	2 (amoxicillin, ceftriaxone)	2
deepseek-v4-pro	C	0	0
llama3.1:8b	B	0	0
llama3.1:8b	C	1 (against its own wrongly extracted maximum; not against the scenario's)	0

Among the three frontier models the check flags exactly the seven uncapped outputs in workflow B and the four in workflow C, and nothing else; agreement between the own-maximum and scenario-maximum columns is complete for them; the single small-model flag is against a maximum the model extracted wrongly. The check is retrospective and has not been tested as a prospective safeguard; it found the seventh uncapped output (64.65 mg) that the tolerance passed.

10.9 tools/export_results.py. Purpose: export of every quoted number and its provenance. Inputs: the scored files, analysis outputs, LOCK.json, DEVIATIONS.md and config/models.yaml. Rules: writes results/export/ with summary.json (every quoted number), tables as CSV, the run manifest (model ids, dates, settings, prompt hashes, cost), LOCK.json, DEVIATIONS.md and a sanitised configuration with environment-variable names only. Output: the results/export/ directory. Usage:

```
python -m tools.export_results 2>&1 | tail -n 5; ls results/export
```

Findings: not a results script; its output is the hand-off set and is refreshed with the final version after Gemini completes.

10.10 tools/code_errors.py. Purpose: blinded manual coding and re-coding of residual extraction errors. Inputs: the run1 scored files and scoring/codebook.md. Rules: collects residual extraction errors from the scored files, shuffles them with a fixed seed, hides the model name, shows the note, the output and the automatic codes, and appends each typed code to results/manual_codes.jsonl immediately; quitting and rerunning resumes. The --recode option draws a 10 percent sample with a second seed, presents it again, and writes agreement to

results/manual_codes_recode.jsonl. The author codes alone; no assistant suggests a code for any item. Output: results/manual_codes.jsonl and results/manual_codes_recode.jsonl. Usage:

```
python -m tools.code_errors --scored 'results/scored_test_*_run1.jsonl'
```

The re-code after two weeks is the same command with --recode 0.10. Findings to date: a first 40-item pass carried assistant input (one suggested code and one suggested reclassification) and was discarded, with the file retained as results/manual_codes_assisted_pass_discarded.jsonl; the author codes all 288 residual errors plus Gemini's unassisted; intra-rater agreement will be reported on the unassisted file. H4 is provisional until this coding is complete.

11. Figures

The figure map and the data file behind each, from the study report.

Figure	File	Data source
1 Study pipeline	F2D_Figure_Study_Pipeline.png	Protocol v1.0; LOCK.json
2 Two workflows	F2D_Figure_Two_Workflows.png	Protocol Section 12; harness configuration
3 Primary endpoint	F2D_Figure_Primary_Endpoint.png	analysis_primary_four_models.txt
4 Small model failure composition	F2D_Figure_Small_Model_Failure_Composition.png	scored run1 files, Llama B and C
5 Heat map, workflow B (report only)	F2D_Figure_Heatmap_Workflow_B.png	by_stratum_run1.csv
6 Heat map, workflow C (report only)	F2D_Figure_Heatmap_Workflow_C.png	by_stratum_run1.csv

Figures 3, 5 and 6 are regenerated with the Gemini row when its runs complete. The figures go into the evidence packet as PNG at 200 dpi with the data file behind each.

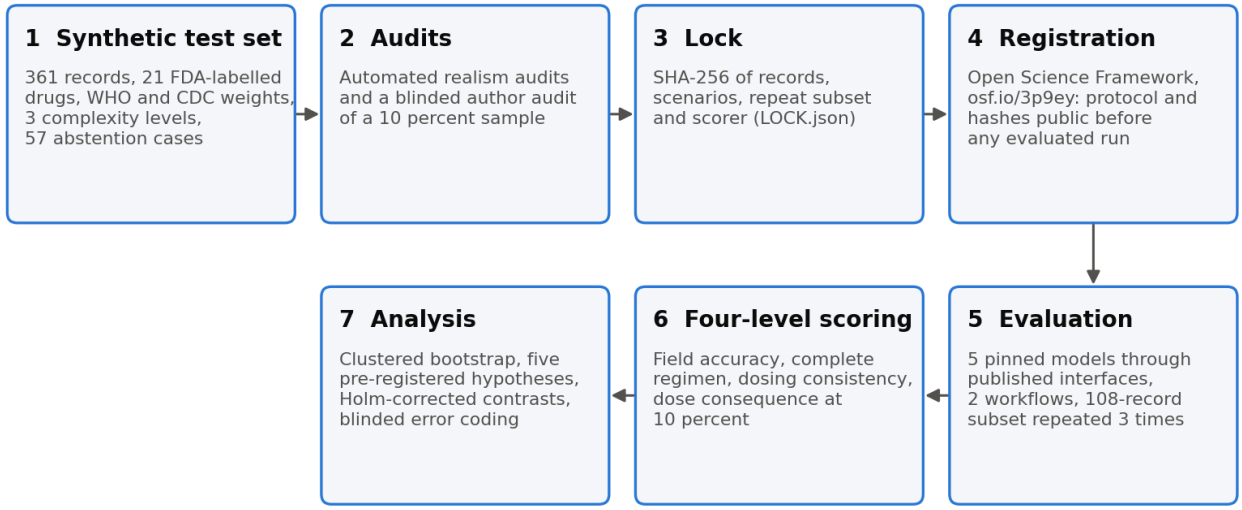


Figure 1. The study in seven steps.

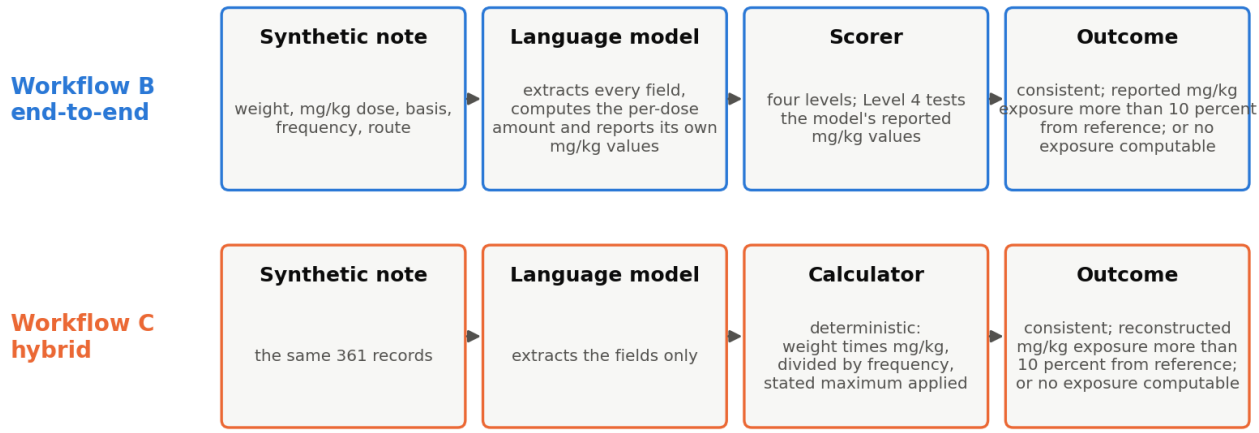


Figure 2. The two workflows and the composite outcome.

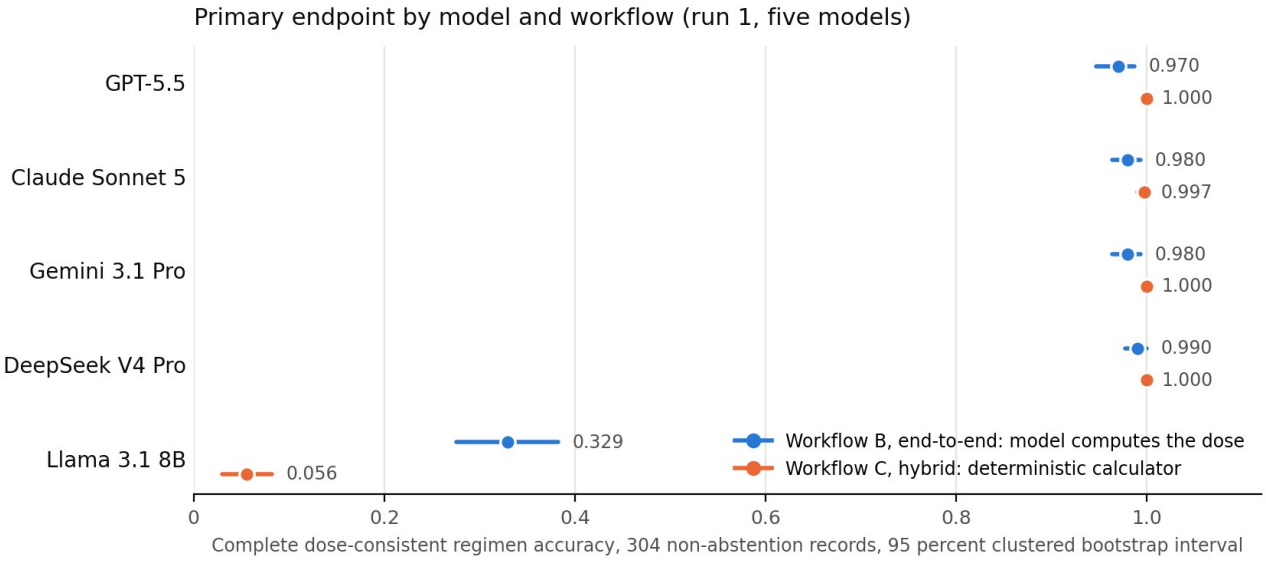


Figure 3. Primary endpoint by model and workflow with 95 percent clustered bootstrap intervals.

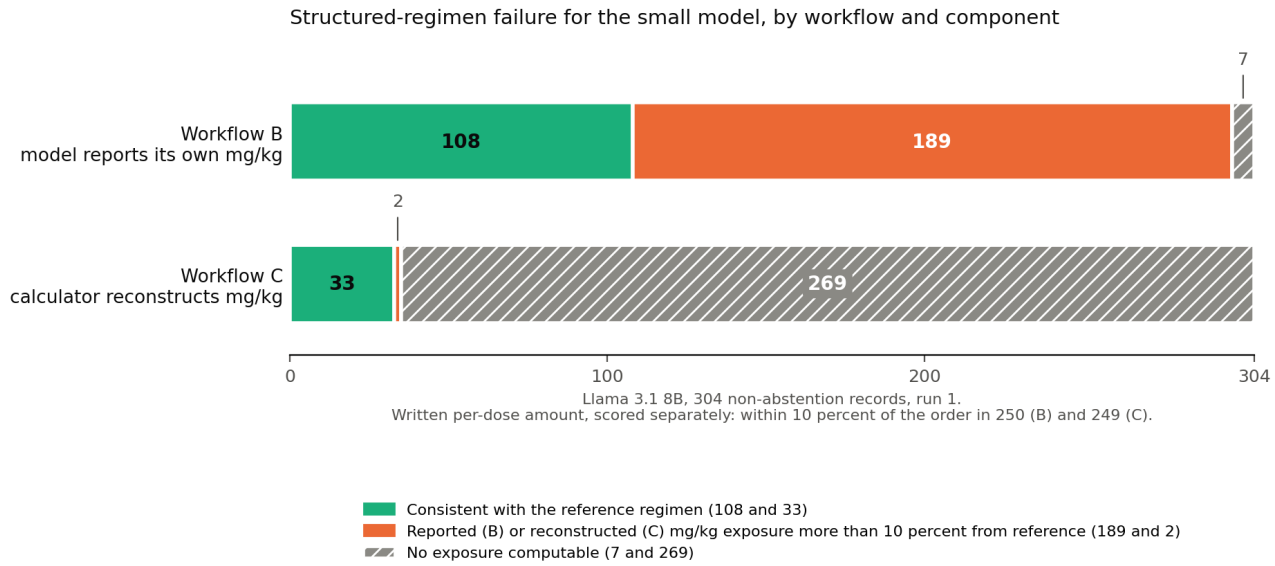


Figure 4. Composite failure for the small model by workflow and component.

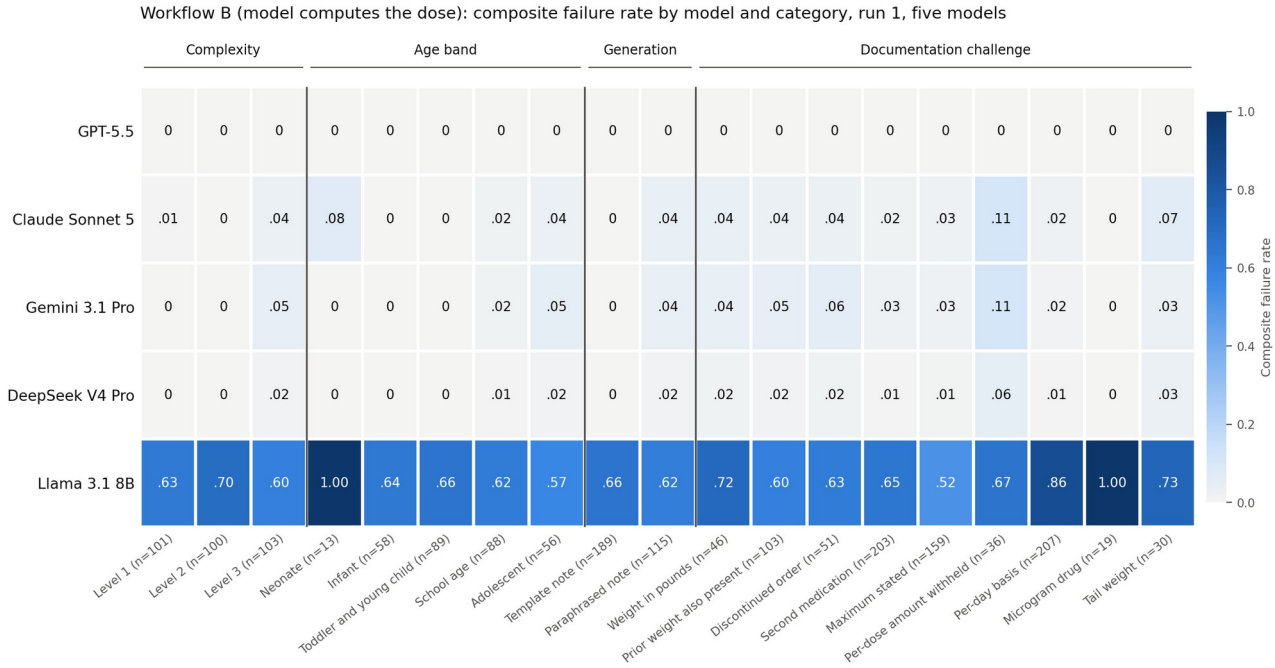


Figure 5. Workflow B composite failure rate by model and category.

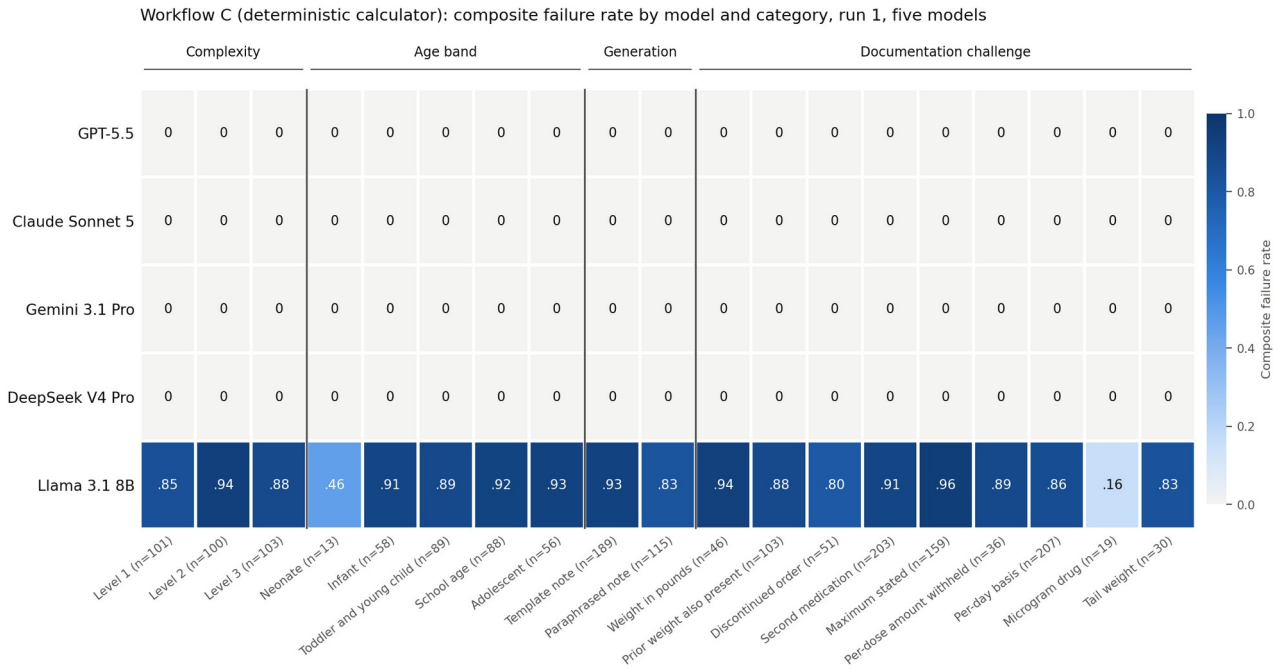


Figure 6. Workflow C composite failure rate by model and category.

12. Evidence packet

The evidence packet lives at results/packet in the public repository. It is built by script from the working repository (31 files at the 5 October build) and refreshed from the final export. The script that builds it is not named in the sources used here. Contents by folder as built:

Folder	Contents
01_lock_verification.txt	LOCK.json, shasum of the three data files, hash_scoring() output, the empty list of post-lock commits to hashed files, HEAD
02_row_level/	stale_weight, maximum_dose, neonatal_rounding: note text, scenario fields, every model's parsed output and scored row in both workflows, as text and JSONL
03_written_amount/	admin_amount, by_stratum, per_drug and max_check CSVs with their scripts and the written-amount versus composite cross-tab
04_h3/	locked and corrected primary analyses, pairwise contrasts, H3 under both wordings
05_manifest/	per-file model, endpoint, workflow, run, attempts, first and last timestamps, prompt hash; Ollama digest; models.yaml with environment-variable names only
06_deviations/	DEVIATIONS.md, LOCK_hashes.txt

The row-level traces in 02_row_level/ are produced by a short script that joins a record's note text, scenario fields, the model's last parsed output and its scored row; it was used for the four maximum-dose notes and the neonatal rounding record. The reconciliation in 03_written_amount/ is a cross-tab of admin_amount_run1.csv status against the scored composite and its component, with the automatic codes among the disagreeing rows.

Still to add with the final version: the final analysis tables with model, workflow, numerator, denominator, CI and adjustment method for every quoted number; the hypothesis ledger with the analysis version; the five stale-

weight records with each model's weight output per workflow and the generator's withheld reference weight; the four maximum-dose notes with all six outputs, scored rows and label set identifiers; the neonatal rounding record with the three models' outputs, scored rows and the scorer's Level 4 path; calculator traces for two workflow B and two workflow C failures, from note to extracted fields to calculator input to result to scored outcome, with "no exposure computable" separated from "exposure more than 10 percent off"; the rounding audit listing the 53 records and the one above 10 percent; registration evidence (dated capture of the OSF registration page showing the hashes, LOCK.json, LOCK_hashes.txt, the shasum and hash_scoring() output, the git log of score.py); source provenance (dosing_sources.csv with label set ids, growth file download record, age_bands.json); figures as PNG at 200 dpi with their data files; approved quotes with the author's written approval.

13. Public release

13.1 What is public. tools/make_public_release.py copies an allow-list from the working repository into a fresh directory: README, licences and CITATION.cff; config/models.yaml (environment-variable names only); sources/; generator/, harness/ (prompts, schema, runner, client), scoring/, analysis/, tools/, tests/; data/test/ (locked split, LOCK.json, repeat subset, audits) and data/dev/; protocol/ (Protocol v1.0 docx and PDF, LOCK_hashes.txt, DEVIATIONS.md, README) without the archive; results/ raw and scored run files, analysis outputs, supplementary CSVs, manual coding files and the evidence packet, without logs and backups; docs/ with the official documents and figures; SETUP_GUIDE.md. Its docstring, verbatim:

```
"""Build the public release directory from the working repository (allow-list copy; see System
Architecture, Section 7).



Copies only listed paths into ../from-fields-to-dose-public (fresh directory, no git history),
writes README, licences and

CITATION.cff, runs a secret pattern scan, and prints a summary. It never touches the working
repository.



Usage (from the working repository root): python -m tools.make_public_release [--out ../from-fields-
to-dose-public]

"""
```

13.2 The allow-list, exclusions and secret patterns, verbatim from the script:

```
ALLOW_DIRS = ["config", "sources", "generator", "harness", "scoring", "analysis", "tools", "tests",
"data/test", "data/dev", "protocol", "docs", "results/packet", "results/export"]

ALLOW_FILES = ["SETUP_GUIDE.md", "README.md", "requirements.txt", "pyproject.toml", ".gitignore"]

RESULT_GLOBS = ["raw_test_*.jsonl", "scored_test_*.jsonl", "*.csv", "analysis_*.txt",
"per_drug_run1.txt", "admin_amount_run1.txt", "by_stratum_run1.txt", "manual_codes*.jsonl"]

EXCLUDE_ANY = ["__pycache__", ".DS_Store", ".env", ".venv", "archive", "logs", "backup", "*.pyc",
"*.log", "secret*"]
```

```
SECRET_PATTERNS = [r"sk-[A-Za-z0-9_\-]{16,}", r"sk-ant-[A-Za-z0-9_\-]{16,}", r"AIza[0-9A-Za-z_\-]
{30,}", r"xai-[A-Za-z0-9]{16,}", r"(?i)api[_-]?key\s*[:=]\s*['\"][A-Za-z0-9_\-]{16,}", r"(?i)bearer\
s+[A-Za-z0-9_\-\.]{20,}"]
```

How the script applies them. The output directory (default ../from-fields-to-dose-public, overridable with --out) is deleted if it exists and recreated. Each ALLOW_DIRS entry that exists is walked with os.walk; any path whose components match an EXCLUDE_ANY pattern (fnmatch on each component) is skipped, and excluded subdirectories are pruned from the walk, so protocol/archive, results/logs and results/backup never enter the copy even though their parents are allowed. Each ALLOW_FILES entry that exists is copied. Top-level files of results/ are copied only when their name matches a RESULT_GLOBS pattern and is not excluded; this is how the raw and scored run files, the CSVs, the analysis text outputs and the manual coding files reach the release while logs and backups do not. The script then writes README.md, LICENSE, LICENSE-DATA, CITATION.cff and a .gitignore (.env, .venv/, __pycache__/, *.pyc, .DS_Store) from its own constants.

13.3 The secret scan. After copying, the script walks the release directory, skips files ending in .png, .pdf, .docx or .zip, reads every other file as text with errors ignored, and searches for each SECRET_PATTERNS regular expression. Any hit is printed as (relative path, first 12 characters of the match followed by "...") and the script exits with status 2 and the message "SECRET SCAN: possible secrets found, do not push:"; with no hits it prints "secret scan: clean". tools/secret_scan.py is run separately before any push of the working repository, including over results and logs.

13.4 What is not public. .env and any environment file, the virtual environment, caches, .DS_Store, results/logs, results/backup, protocol/archive, and any file not on the allow-list.

13.5 The fresh-history rule. The release directory has its own git history with no connection to the working repository's history, so nothing excluded can be recovered from it. The script never touches the working repository.

13.6 Licences and CITATION.cff. The README constant states the verification chain (the lock, the recompute commands, the deviations log, the packet), the layout, the reproduce commands for the headline tables, the status and the licences, and carries the ORCID, the registration (osf.io/3p9ey, DOI 10.17605/OSF.IO/3P9EY) and the project (osf.io/uajs3). Code is released under MIT (LICENSE). Synthetic data, protocol, outputs and documents are released under CC BY 4.0 (LICENSE-DATA); that file states that the synthetic records contain no real patient data, that FDA label excerpts in sources/labels are public-domain US government works, and that WHO and CDC growth reference tables are redistributed under their own terms. CITATION.cff (cff-version 1.2.0) gives the study title, the author's ORCID, the OSF DOI as an identifier, the repository URL, the MIT licence and type software. The licence and citation constants carry the author's name and are not reproduced here.

13.7 Procedure. Run the release script; run tools/secret_scan.py and a pattern scan over the release directory; initialise a fresh repository, commit, push to the public remote; tag the commit; record the tag in the paper. Repeat at each milestone (fifth model complete; coding complete; acceptance). At acceptance: update the OSF project with the paper DOI and the repository URL; rotate all provider keys. The exact commands are in the runbook (Section 14).

14. Operating runbook

Every command block from the operating record, in order of use, exactly as given. Each is preceded by one line on when to run it and what to expect.

14.1 Lock verification (BLOCK 4, after the lock; repeated on 5 October and before any release). Expect the three shasum values to equal records_sha256, scenarios_sha256 and repeat_subset_sha256 in LOCK.json, the printed hash_scoring() value to equal scoring_sha256 (7be9bc8704e09b94014741847f16b047b115c7b2c7c7846ec01afe03445461ed), and the git log to print nothing.

```
shasum -a 256 data/test/records.jsonl data/test/scenarios.jsonl data/test/repeat_subset.txt

python3 -c "from tools.lock_testset import hash_scoring; print(hash_scoring())"

git log --format='%h %ad %s' --date=iso 528c2b5..HEAD -- scoring/score.py generator/calculator.py
harness/prompts/system.txt harness/prompts/task_B.txt harness/prompts/task_C.txt
harness/output_schema.json analysis/analyze.py
```

14.2 Gemini chain (BLOCK 6; one project; rerun unchanged after the key swap for the second project). Expect a background job writing to results/logs/gemini_evening1.log; the tail shows the runner's first progress lines after a minute; if the first project's quota is exhausted the model loop stops with a quota message and the chain is rerun after 14.4.

```
    PYTHONUNBUFFERED=1 caffeinate -i nohup zsh -c 'python -m harness.runner --split test --workflow
B --run-id run1 --model gemini-3.1-pro-preview; for w in C B; do for r in run2 run3; do python -m
harness.runner --split test --workflow $w --run-id $r --subset data/test/repeat_subset.txt --model
gemini-3.1-pro-preview; done; done' > results/logs/gemini_evening1.log 2>&1 &

    sleep 60; tail -n 3 results/logs/gemini_evening1.log
```

14.3 Status of runs (any time during BLOCK 6). Expect one line per raw file with the count of records with parsed output against 361 for run1 files and 108 for repeat files, DONE where the target is reached, and a final line stating whether a runner process is alive.

```
    python3 - <<'PY'

    import json,glob

    for f in sorted(glob.glob("results/raw_test_*_run1.jsonl"))+sorted(glob.glob("results/
raw_test_*_run[23].jsonl")):

        ok=set()

        for l in open(f):

            o=json.loads(l)

            if "record_id" in o and o.get("parsed") is not None: ok.add(o["record_id"])

        target=361 if "run1" in f else 108

        print(f"{len(ok):4d} / {target}  {'DONE' if len(ok)>=target else '':5s} {f.split('/')[-1]}")

    PY

    pgrep -f harness.runner >/dev/null && echo "runner: RUNNING" || echo "runner: STOPPED"
```

14.4 Key swap between the two Google Cloud projects (when the Gemini loop stops on the per-project quota; both keys kept in .env, one active). Expect the message "swapped; active key now ends with" followed by the last four characters of the newly active key; then rerun 14.2.

```
    python3 - <<'PY'
```

```
    L=open(".env").read().splitlines(); d={l.split("=",1)[0]:i for i,l in enumerate(L) if "=" in l}

    a,b=d["GOOGLE_API_KEY"],d["GOOGLE_API_KEY_PROJECT1"]; va=L[a].split("=",1)[1];
vb=L[b].split("=",1)[1]

    L[a]="GOOGLE_API_KEY="+vb; L[b]="GOOGLE_API_KEY_PROJECT1="+va; open(".env","w").write("\
n".join(L)+"\n"); print("swapped; active key now ends with",vb[-4:])

    PY
```

14.5 Final scoring and analysis (BLOCK 7 and BLOCK 8, after all runs complete). Expect the scorer's last two lines and a count of 30 scored files; analysis_primary.txt, analysis_stability.txt with ten model and workflow sections, analysis_pairwise.txt, per_drug_run1.txt, admin_amount_run1.txt and by_stratum_run1.txt written; max_check_run1.csv written; the export's last five lines and a listing of results/export. The scorer exits before scoring if the lock check fails.

```
    python -m scoring.score --raw results/raw_test_*.jsonl 2>&1 | tail -n 2; ls
results/scored_test_*.jsonl | wc -l

    python -m analysis.analyze --scored 'results/scored_test_*_run1.jsonl' | tee
results/analysis_primary.txt > /dev/null

    rm -f results/analysis_stability.txt; for m in gpt-5.5-2026-04-23 claude-sonnet-5 gemini-3.1-
pro-preview deepseek-v4-pro llama3.1_8b; do for w in C B; do echo "## $m wf=$w" >>
results/analysis_stability.txt; python -m analysis.stability --scored results/scored_test_${w}_$
{m}_run1.jsonl results/scored_test_${w}_${m}_run2.jsonl results/scored_test_${w}_${m}_run3.jsonl >>
results/analysis_stability.txt 2>&1; done; done

    python -m analysis.pairwise --scored 'results/scored_test_*_run1.jsonl' | tee
results/analysis_pairwise.txt

    python -m tools.per_drug --scored 'results/scored_test_*_run1.jsonl' > results/per_drug_run1.txt

    python -m tools.admin_amount --scored 'results/scored_test_*_run1.jsonl' | tee
results/admin_amount_run1.txt

    python -m tools.by_stratum --scored 'results/scored_test_*_run1.jsonl' | tee
results/by_stratum_run1.txt

    python -m tools.max_check

    python -m tools.export_results 2>&1 | tail -n 5; ls results/export
```

14.6 Manual coding (BLOCK 7, after scoring). Expect an interactive session that presents each residual error blinded and in shuffled order and appends each typed code to results/manual_codes.jsonl; quitting and rerunning resumes. The re-code after two weeks is the same command with --recode 0.10, which draws a 10 percent sample with a second seed and writes agreement.

```
python -m tools.code_errors --scored 'results/scored_test_*_run1.jsonl'
```

14.7 Public release (after export, at each milestone). Expect the release directory ../from-fields-to-dose-public rebuilt with README, LICENSE (MIT), LICENSE-DATA (CC BY 4.0), CITATION.cff and .gitignore, a count of files copied, and "secret scan: clean"; on any hit the script exits with status 2 and nothing is pushed. Then, in that directory, the fresh repository is initialised and pushed; the commit is tagged and the tag recorded in the paper.

```
python -m tools.make_public_release
```

```
git init -q; git add -A; git commit -q -m "..."; git branch -M main; git remote add origin
https://github.com/VinodRufus/from-fields-to-dose.git; git push -u origin main
```

15. Known limitations of the implementation

From the architecture document. A single author operates every stage; there is no clinician or pharmacist review, and the ground truth is by construction. The realism auditor and the paraphrase generator are the same model family (qwen2.5:14b), so the audit is not independent of the paraphrase. The prompt omits a staleness rule for documented weights; this is now a reported finding (all completed frontier models accepted a three-month-old prior-visit weight as current on the same five records). Preview model identifiers may be withdrawn by vendors, and the DeepSeek identifier resolves to the provider's current release at call time, so the manifest records call timestamps rather than claiming immutability. Local inference for Llama is deterministic at the chosen settings, so its stability figure is not comparable to the API models. Resumed runs give unparsed outputs an extra attempt, applied uniformly and reported.

Level 4 source in workflow B. Workflow B's Level 4 trusts the model's reported mg_per_kg_per_dose and mg_per_kg_per_day fields. A model that copies the printed amount correctly can still fail on its own inconsistent mg/kg fields, which is why the written amount is reported separately (admin_amount.py), and a model whose amount field and mg/kg fields disagree with each other can pass both Level 2 and Level 4, as the neonatal probe demonstrated; the scorer has no consistency check between the written amount and the reported exposure.

The per-day-only flag. exceeds_labeled_max tests the observed daily exposure against max_per_day_mg only; per-dose maxima are not tested, so the amoxicillin 875 mg per dose outputs carry the flag False. The flag does not feed the primary endpoint. In workflow C the calculator applies the cap, so an uncapped written amount in that workflow is invisible both to the flag and to Level 4; the post hoc maximum check is the only analysis that sees it, and that check is retrospective and untested as a prospective safeguard.

The rounding record. One test-set record (neonatal dexamethasone) has a printed amount more than 10 percent from the exact product, and about 53 of 304 are more than 1 percent from it. On the one record no internally consistent per-dose amount meets both the 1 percent Level 2 tolerance and the 10 percent Level 4 tolerance in workflow B. The test set and scorer are locked and applied as registered; the record is disclosed, removed by the 20 percent sensitivity, and a Level 2 sensitivity against the exact product is a listed follow-up.

The tolerance passing a cap violation. One Claude workflow B output (64.65 mg against a 60 mg per day maximum, 7.75 percent over) exceeded a stated maximum and passed Level 4 because it is inside the 10 percent tolerance; the per-day flag recorded it but the endpoint did not. The composite is an operational threshold on a constructed intended dose, not a harm measure, and an output can violate a written maximum while remaining dose-consistent under it.

Other implementation notes. The exact columns of max_check_run1.csv, the bootstrap seed of analyze.py, the usage line of analyze_malformed_as_failure.py, the names of the non-Google key variables and the script that builds the evidence packet are not recorded in the sources used for this document.

Final results with the fifth model

Gemini 3.1 Pro completed all runs on 6 October (run 1 on 361 records in both workflows; three runs on the 108-record subset; zero errors in the final chains), across two Google Cloud projects under one billing account after

the per-project daily quota. The final scoring and analysis were run on the full set of 30 files. What changed and what did not:

Item	Final value
Gemini 3.1 Pro, end-to-end	field accuracy 1.000; complete regimen 0.997; composite failure 0.016 (5 of 304); primary endpoint 0.980 (0.964 to 0.993)
Gemini 3.1 Pro, hybrid	all 1.000; composite failure 0
Gemini stability	1.000 (hybrid), 0.991 (end-to-end)
Pairwise	no significant difference among the four frontier models in either workflow (largest adjusted p 0.92 is the smallest); every frontier model exceeds Llama by 0.641 to 0.661 (end-to-end) and 0.941 to 0.944 (hybrid)
Abstention	all four frontier models miss the same five stale-weight records in both workflows, writing the prior weight as current
Maximum-dose outputs	nine over five notes (Claude 5, DeepSeek 2, Gemini 2); seven beyond the tolerance; amoxicillin per-dose maximum missed by three of four frontier models; GPT-5.5 applied the maximum in all five
New: reporting-convention finding	Gemini capped the written amount correctly in four notes and reported the order's nominal mg/kg rather than the capped figure; the prompt does not state the convention; scored as registered (four composite failures with correct written amounts) and disclosed as a clarification
H3 under Section 1 (composite rate)	Claude -0.016, Gemini -0.016 supported; GPT-5.5 null; DeepSeek -0.007 not; Llama +0.247 reversed; 2 of 5, not supported
H3 under Section 16 (primary endpoint)	GPT-5.5 +0.030, Claude +0.016, Gemini +0.020 supported; DeepSeek +0.010 not; Llama -0.273 reversed; 3 of 5, supported
Frontier composite failures, end-to-end	twelve: Claude 5, DeepSeek 2, Gemini 5; all in paraphrased notes
Written-amount deviations, end-to-end	ten: GPT-5.5 1, Claude 4, DeepSeek 3, Gemini 2
Maximum check denominator	3,040 eligible first-run outputs; nine end-to-end and four hybrid flags among the frontier models; one small-model flag against its own wrong maximum
Figures	3, 5 and 6 regenerated with the Gemini row

Still pending: unassisted manual coding and the re-code; TRIPOD-LLM checklist; dated registration capture; approval of any quotation; public repository refresh from the final export; key rotation at acceptance.

Repository

Code, synthetic data, locked test split, protocol, raw and scored outputs, analysis outputs, evidence packet, figures and documents: https://github.com/VinodRufus/from-fields-to-dose